

3DGEER: 3D GAUSSIAN RENDERING MADE EXACT AND EFFICIENT FOR GENERIC CAMERAS

Anonymous authors
Anonymous submission

ABSTRACT

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) attains a favorable trade-off between reconstruction fidelity and computational throughput, yet its efficiency is predicated on a local affine approximation that maps each 3D Gaussian to a 2D image-plane projection. This assumption induces a systematic bias that grows with the nonlinearity of the projection function, and is therefore most pronounced under generic large field-of-view (FoV) cameras. To date, no prior formulation attains *projective exactness* and *real-time efficiency* simultaneously for arbitrary camera models. We introduce 3DGEER, a geometrically exact and computationally efficient Gaussian rendering framework. Proceeding from first principles, we derive a closed-form expression for the transmittance integral of Gaussian density along a ray, which enables exact forward rendering and gradient-based optimization of the Gaussian parameters under arbitrary camera models. To preserve efficiency, we introduce two components: the *Particle Bounding Frustum* (PBF), an analytic construction that yields tight ray-Gaussian association without bounding-volume-hierarchy (BVH) traversal; and the *Bipolar Equiangular Projection* (BEAP), an image parameterization that unifies FoV representations across camera models, accelerates the association stage, and yields a more uniform ray-sampling distribution that improves reconstruction quality. Empirically, on both pinhole and fisheye datasets, 3DGEER surpasses prior methods across all reported metrics, executes $5\times$ faster than existing projective-exact ray-based baselines, and generalizes to FoVs wider than those observed during training, thereby establishing a new state of the art in real-time radiance field rendering.

1 INTRODUCTION

Initiated by 3D Gaussian Splatting (3DGS) (Kerbl et al., 2023), volumetric particle rendering methods cast in the splatting formulation (Yu et al., 2024a; Yan et al., 2024; Lu et al., 2024; Kheradmand et al., 2024; Charatan et al., 2024; Mallick et al., 2024) have been widely adopted on account of their high reconstruction fidelity and rendering throughput. This efficiency is inherited from EWA splatting (Zwicker et al., 2001), which applies a first-order (local affine) approximation of the projection map to render each 3D Gaussian as a 2D Gaussian on the image plane. The approximation is accurate under narrow field-of-view (FoV) conditions, where the projection is nearly linear over the support of a Gaussian; however, it incurs substantial error in wide-FoV regimes, in which the nonlinearity of the projection causes the true image-plane footprint to depart markedly from a symmetric 2D Gaussian. This regime is characteristic of fisheye imagery in robotics and autonomous driving (Rashed et al., 2021; Courbon et al., 2007; Kumar et al., 2023; Zhang et al., 2016; Yogamani et al., 2019; 2024; Sekkat et al., 2022). Recent methods extend differentiable splatting to specific fisheye camera models (Liao et al., 2024) or to broader camera families (Huang et al., 2024b), but they remain fundamentally bound to approximations formulated in 2D projective space and do not admit an exact closed-form solution.

A complementary line of reasoning, advanced by Celarek et al. (2025), questions the necessity of exact rendering, arguing that increasing the Gaussian count (up to 1M) suffices to suppress the projective approximation error in splatting. This evidence, however, is confined to small object-centric scenes. In large-scale reconstruction the requisite Gaussian density varies sharply across objects and frustums, rendering uniform scaling to larger scenes prohibitively expensive. Moreover,

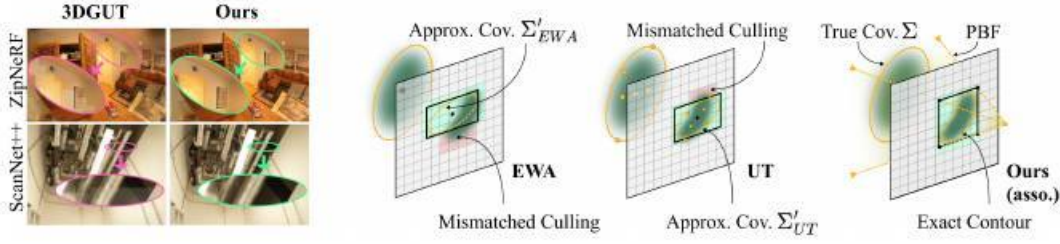


Figure 1: **Linear Approximation Error in Ray-Particle Association.** (Left) Grid-line artifacts induced by inaccurate Unscented-Transform (UT) association. (Right) Schematic of our association relative to prior schemes: rather than fitting an intermediate 2D conic, our method computes the exact bounding structure directly from the true 3D covariance.

we demonstrate that the projective error under generic distorted cameras is substantially larger, such that increasing the Gaussian count alone fails to close the resulting quality gap (see Fig. 5).

A parallel line of work formulates volumetric particle rendering directly as ray tracing or ray marching (Yu et al., 2024b; Condor et al., 2025; Mai et al., 2025; Moënné-Loccoz et al., 2024; Talegaonkar et al., 2025; Gu et al., 2025). Because these formulations integrate density along rays rather than in the image plane, they accommodate large-FoV camera models natively and are free of projective approximation error. Their principal difficulty is efficiency, and specifically the ray-particle association subproblem: identifying, for each rendering ray, the set of 3D particles that contribute to it. Considerable effort has targeted this bottleneck—for instance, the acceleration structures based on Bounding Volume Hierarchies (BVH) employed in EVER (Mai et al., 2025) and 3DGR (Moënné-Loccoz et al., 2024). Owing to the algorithmic complexity of hierarchy traversal and its limited amenability to GPU parallelization, however, pure ray-based methods have not yet matched the throughput of 3DGS. As an alternative, 3DGUT (Wu et al., 2025) adopts a hybrid design that couples ray-based rendering with rasterization-based association, recovering frame rates comparable to 3DGS. Yet the association stage reintroduces a projective approximation error that compromises exactness in projective geometry and can produce grid-line artifacts (see Fig. 1 Left). Consequently, no existing method simultaneously attains the geometric exactness of ray-based rendering and the throughput of splatting.

In this work, we propose 3DGEER, a 3D Gaussian Exact and Efficient Rendering method that attains *exactness in projective geometry* (formalized in Sec. A) while retaining *throughput comparable to splatting* for generic cameras, encompassing both perspective and a range of fisheye projection models. 3DGEER rests on three technical contributions: (i) We reexamine the heuristic maximum-response function (Keselman & Hebert, 2022) for Gaussians from first principles and establish its equivalence to projective exactness; from this equivalence we derive a differentiable rendering framework with provable numerical stability. (ii) To perform ray-particle association efficiently without sacrificing geometric exactness, we recast the problem as an intersection test between Camera Sub-Frustums (CSF) and Particle Bounding Frustums (PBF), directly analogous to the tile-AABB mapping in 3DGS. For each 3D Gaussian we derive a closed-form expression for the PBF that tightly bounds it, which admits highly efficient association. (iii) We further propose to draw color-supervision rays by uniform sampling in a novel Bipolar Equiangular Projection (BEAP) space. This sampling strategy aligns the image-space partition with the underlying CSF partition and, empirically, improves reconstruction quality relative to conventional pinhole or equidistant projections.

A comprehensive evaluation on fisheye and pinhole datasets—ScanNet++ (Yeshwanth et al., 2023), Zip-NeRF (Barron et al., 2023), Aria (Lv et al., 2025), and MipNeRF (Barron et al., 2021)—establishes that 3DGEER consistently outperforms prior and concurrent methods, yielding PSNR gains of 0.9–4.8 dB under challenging wide-FoV conditions and 0.3–2.0 dB on pinhole views. Notably, when trained exclusively on narrow-FoV data, our method still surpasses splat-based baselines on wide-FoV cameras, evidencing strong extrapolation to extremely wide-FoV rendering.

Table 1: **Summary of Particle Rendering Methods w.r.t. Projective Exactness.** The upper block enumerates splatting-based methods; the lower block lists ray-based methods, a subset of which nonetheless perform association through a projective approximation (e.g., EWA or the Unscented Transform). The columns record, for each method, its rendering formulation, its association method and scope, its particle primitive, and whether it supports large FoV, high frame rates, and exact rendering.

Method	Render	Particle Asso. Method Scope	Particle Type	Large FoV	High FPS	Exact Meth.
3DGS [Kerbl et al., 2023]	Splat.	EWA Img.	3D Gauss.	✗	✓	✗
FisheyeGS [Liao et al., 2024]	Splat.	EWA Img.	3D Gauss.	✓	✓	✗
GS++ [Huang et al., 2024b]	Splat.	EWA Img.	3D Gauss.	✗	✓	✗
2DGS [Huang et al., 2024a]	Ray	Splat Bound. Img.	2D Gauss.	✗	✓	✓
GOF [Yu et al., 2024b]	Ray	EWA Img.	3D Gauss.	✗	✗	✗
EVER [Mai et al., 2025]	Ray	BVH Scene	3D Ellip.	✓	✗	✓
3DGRT [Moënné-Loccoz et al., 2024]	Ray	BVH Scene	3D Gauss.	✓	✗	✓
3DGUT [Wu et al., 2025]	Ray	UT Img.	3D Gauss.	✓	✓	✗
HTGS [Hahlbohm et al., 2025]	Ray	Splat Bound. Img.	3D Gauss.	✗	✓	✓
3DGEER (Ours)	Ray	PBF Frust.	3D Gauss.	✓	✓	✓

2 RELATED WORKS

2.1 SPLATTING-BASED GAUSSIAN RENDERING

The majority of 3DGS variants [Yu et al., 2024a, Zhang et al., 2024, Rota Bulò et al., 2024, Gao et al., 2025, Zhao et al., 2025, Kerbl et al., 2024, Hou et al., 2025, Liu et al., 2025] inherit the EWA splatting operator [Zwicker et al., 2001, Ren et al., 2002], which projects 3D Gaussians onto the image plane and represents their footprint by 2D conics. To enable differentiable rendering under distorted large-FoV camera models, methods such as FisheyeGS [Liao et al., 2024] and GS++ [Huang et al., 2024b] evaluate the Jacobian of the equidistant or spherical projection and use it to correct the 2D mean and covariance of the projected Gaussian. These procedures nevertheless retain a first-order Taylor expansion of a strongly nonlinear projection map. In wide-FoV regimes the neglected higher-order terms are non-negligible, so the residual approximation error is significant and degrades reconstruction quality.

The underlying obstacle is that, under any nonlinear camera model—whether an ideal pinhole or a distorted projection—the exact projection of a 3D Gaussian is not a symmetric 2D Gaussian. Equivalently, any framework that approximates an inherently nonlinear transformation by linear projective geometry incurs an irreducible approximation error. Our method sidesteps this obstacle entirely: it integrates Gaussian density directly along rays within a volumetric rendering formulation, thereby introducing no projective approximation.

2.2 RAY-BASED GAUSSIAN RENDERING

Keselman & Hebert (2022) posit that a ray interacts with a Gaussian density at the point of maximum intensity response. Building on this premise, 3DGRT [Moënné-Loccoz et al., 2024] adopts the maximum-response intersection to construct a 3D Gaussian ray-tracing framework that samples directly in ray space, dispensing with explicit projection and thereby mitigating wide-FoV distortion. The cost of 3DGRT is substantial, however: its ray-particle association depends on scene-level spatial partitioning via BVHs, whose construction and traversal overhead yields low frame rates.

EVER [Mai et al., 2025] instead replaces the Gaussian primitive with a constant-density ellipsoid, for which transmittance-weighted ray integration admits a closed-form solution. This substitution, however, lowers the representational capacity relative to full 3D Gaussians, and runtime remains limited by its dependence on BVH structures. 2DGS [Huang et al., 2024a] substitutes 2D surfels for 3D Gaussians, which permits exact ray-surface intersection. HTGS [Hahlbohm et al., 2025] restores the full Gaussian primitive while retaining the ray-splat association of 2DGS; because that association is confined to screen space, however, it does not guarantee projective exactness at wider FoVs. Our contribution, by contrast, derives closed-form bounds and an end-to-end rendering pipeline directly in a camera-agnostic angular domain (Sec. D); we analyze the relation to, and the computational differences from, the 2DGS and HTGS parameterizations and rendering procedures in Sec. D.3

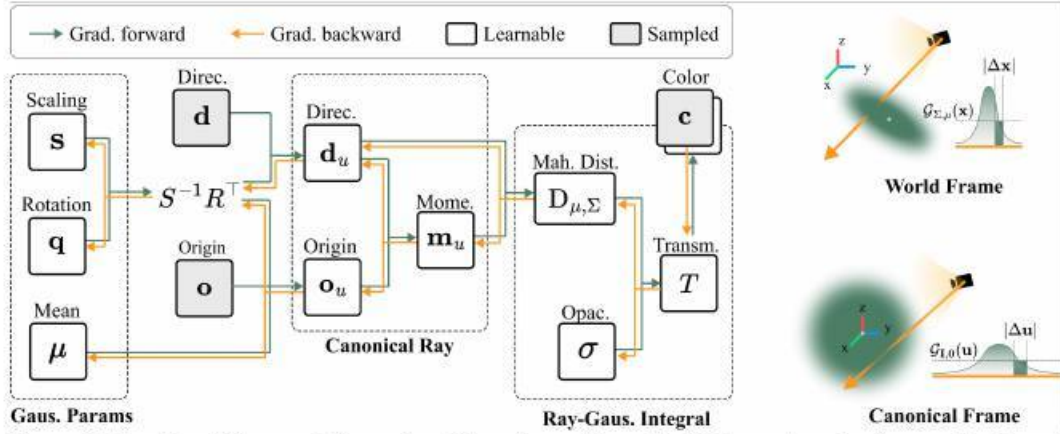


Figure 2: **Gradient Flow and Canonical Transformation.** (Left) Forward evaluation and backward gradient propagation linking the Gaussian parameters s , q , μ and the opacity coefficient σ to the transmittance. (Right) The transmittance integral of 3D Gaussian density along a ray; the green box denotes the elementary contribution, i.e., the product of density and ray-segment length (see Eq. B.5).

3DGUT (Wu et al. 2025), a successor to 3DGRT, improves throughput by adopting splatting-based particle association, reaching frame rates comparable to 3DGS. Its acceleration, however, rests on the Unscented Transform (UT)—a sampling-based approximation that estimates 2D conics in projective image space. GOF (Yu et al. 2024b) exhibits an analogous limitation, applying the EWA construction to fit 2D bounding conics and projected centers for association. Under extreme-FoV conditions, the resulting imprecise axis-aligned bounding-box (AABB) estimates yield inaccurate tile-to-AABB assignments and produce visible grid-line artifacts (see Fig. 1 Left).

In contrast, our method resolves ray-particle association at the frustum level directly in 3D space (see Fig. 1 Right), preserving the geometric exactness of ray-based rendering for arbitrary camera models while sustaining throughput comparable to 3DGS. Tab. 1 summarizes representative methods addressing the projective approximation error and contrasts them along several salient axes.

3 METHOD

We present 3DGEER, a Gaussian rendering framework that attains geometric exactness and real-time efficiency jointly under arbitrary-FoV cameras. The framework resolves the two principal bottlenecks of prior work—the *projection approximation* of splatting-based rendering and the *BVH overhead* of ray-based rendering—through three complementary components:

- Ray-based rendering formulation (Sec. 3.1): a closed-form, differentiable solution for the integral of Gaussian density along a 3D ray, which eliminates the linearization error of projection-based splatting.
- Particle Bounding Frustum (PBF) (Sec. 3.2): a ray-particle association scheme that replaces BVH traversal with an exact frustum-particle intersection test, delivering accuracy and efficiency simultaneously.
- Bipolar Equiangular Projection (BEAP) (Sec. 3.3): an image representation that unifies arbitrary-FoV camera models, accelerates PBF computation, and improves reconstruction quality.

The Appendix supplies: (i) the preliminaries and notation (Sec. B.1); (ii) the complete proof accompanying Sec. 3.1 which establishes the equivalence between the maximum-response heuristic and projective exactness (Sec. B); (iii) the full backward-gradient derivations together with their numerical-stability analysis (Sec. C); and (iv) the full derivation of the geometrically exact association (Sec. D).

3.1 RENDERING WITH EXACT PROJECTIVE GEOMETRY

The central challenge in geometrically exact Gaussian rendering is to obtain a closed-form expression for the integral of density and color along a ray. Our approach maps each anisotropic world-space Gaussian into a canonical coordinate system in which every Gaussian is rendered isotropic. Under this mapping the transmittance integral reduces to a measure-preserving change of variables, which admits a compact closed-form solution.

Canonical transformation. For a Gaussian parameterized by mean μ , rotation R , and scaling S , we define the affine mapping

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (1)$$

which sends a canonical coordinate $\mathbf{u}$ to its world coordinate $\mathbf{x}$. Differentiating gives $\Delta \mathbf{x} = RS\Delta \mathbf{u}$, so the Jacobian determinant is absorbed into the density measure (see Fig. 2 Right). Under this change of variables every Gaussian collapses to the shared isotropic form

$$\mathcal{G}_{\mathbf{I},0}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\|\mathbf{u}\|^2\right). \quad (2)$$

Closed-form transmittance. A world-space ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ maps to the canonical ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$, whose parameters are

$$\mathbf{o}_u = S^{-1}R^\top(\mathbf{o} - \mu), \quad \mathbf{d}_u = S^{-1}R^\top\mathbf{d}. \quad (3)$$

The accumulated transmittance then evaluates to

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I},0}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2}D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d})\right), \quad (4)$$

where

$$D_{\mu,\Sigma}^2(\mathbf{o}, \mathbf{d}) = \frac{\|\mathbf{o}_u \times \mathbf{d}_u\|^2}{\|\mathbf{d}_u\|^2} \quad (5)$$

is the squared perpendicular (Mahalanobis) distance from the ray to the Gaussian center.

Discussion. This result shows that the transmittance is a function of a single scalar—the ray-to-center distance in canonical space—which yields an efficient, geometrically exact closed-form solution. The expression coincides with the “maximum-response” heuristic of prior work (Keselman & Hebert [2022]; Moënné-Loccoz et al. [2024]); our derivation, however, exposes the projective exactness that underlies it (Sec. B). Moreover, evaluating both the forward closed form and the associated backward gradient chain analytically (Fig. 2 Left; Sec. C) circumvents the numerical instabilities reported for prior ray-based methods (Yu et al. [2024b]) and removes the need for auxiliary post-processing such as filtering degenerate Gaussians.

3.2 RAY-PARTICLE ASSOCIATION WITH EXACT PROJECTIVE GEOMETRY

For any ray-based Gaussian renderer, both accuracy—most acutely under large-FoV camera models—and runtime are governed by the ray-particle association strategy, a component that prior work has largely neglected. To determine the Gaussians contributing to each ray, we reduce association to an intersection test between Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs), where a PBF is the tight frustum enclosing a Gaussian at a prescribed distance from its center (Fig. 3). A Gaussian is assigned to a CSF whenever its PBF intersects that CSF, and every Gaussian assigned to a CSF is treated as a contributor to all rays within it.

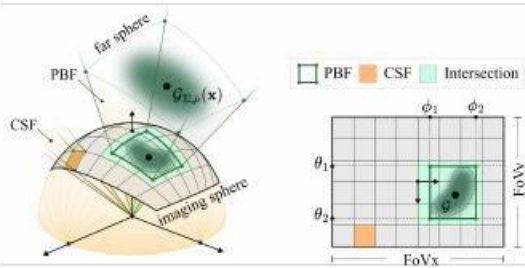


Figure 3: **PBF-CSF Association.** (Left) The PBF specified by four planes tangent to the 3D Gaussian ellipsoid. (Right) The PBF-CSF intersection, unfolded onto the BEAP imaging plane.

To compute a PBF efficiently, we first introduce two spherical angles (see Fig. E.1) in camera space:

$$\theta = \arctan\left(\frac{\mathbf{d}_{c,x}}{\mathbf{d}_{c,z}}\right), \quad \phi = \arctan\left(\frac{\mathbf{d}_{c,y}}{\mathbf{d}_{c,z}}\right), \quad (6)$$

where $(\mathbf{d}_{c,x}, \mathbf{d}_{c,y}, \mathbf{d}_{c,z})$ is the normalized direction toward the camera- (view-) space point $\mathbf{d}_c$. The angles θ and ϕ are the ray's incidence angles in the horizontal (xz) and vertical (yz) planes, each confined to $[-\frac{\pi}{2}, \frac{\pi}{2}]$. We specify a PBF by four planes—two associated with the θ bounds and two with the ϕ bounds (see Fig. 3):

$$\mathbf{g}_{\theta_{1,2}} = (-1, 0, \tan \theta_{1,2}, 0)^\top, \quad \mathbf{g}_{\phi_{1,2}} = (0, -1, \tan \phi_{1,2}, 0)^\top, \quad (7)$$

where the angular limits satisfy $\theta_1 < \theta < \theta_2$ and $\phi_1 < \phi < \phi_2$.

Given a camera with extrinsic matrix $V = [R_c | \mathbf{t}_c] \in \text{SE}(3)$ and a canonical-to-world matrix H (Eq. B.4), we transform the view-space angular-limit planes into canonical space via the inverse transpose of the camera-to-canonical matrix $(VH)^{-1}$; this yields the constraints defining the PBF. Taking the θ bounds as the representative case, the corresponding canonical plane is

$$\mathbf{g}_{u_{1,2}} = (VH)^\top \mathbf{g}_{\theta_{1,2}} = -(VH)_0 + \tan \theta_{1,2} (VH)_2, \quad (8)$$

where $(VH)_i$ denotes the i -th row of the matrix, written as a column vector.

For a Gaussian ellipsoid delimited by a λ -standard-deviation contour, we obtain the angular bounds of the local frustum by imposing tangency of the transformed planes to the ellipsoidal contour, which gives the quadratic constraints

$$((\lambda^2, \lambda^2, \lambda^2, -1) \circ \mathbf{g}_u^\top) \cdot \mathbf{g}_u = 0, \quad (9)$$

where $\circ$ is the Hadamard (element-wise) product and λ controls the contour spread, fixed to 3 throughout. Applying the identical constraint to $\mathbf{g}_v^\top$ yields the corresponding bounds $\phi_{1,2}$. The roots of these quadratics are the tangents of the angular bounds $[\theta_1, \theta_2]$ and $[\phi_1, \phi_2]$, which define the PBF angular limits. Because these quadratics are imposed under the canonical-space constraints, their solution respects the true camera-space 3D covariance Σ_c and mean μ_c of the Gaussian. Concretely, the θ bounds $c = \tan \theta_{1,2}$ satisfy

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad \text{where} \quad \mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (10)$$

This closed form is inexpensive to solve (Sec. D.1) and dispenses with both the intermediate conic approximations of EWA and UT and the screen-space constraints of HTGS and 2DGS that restrict their admissible FoV.

3.3 BIPOLAR EQUIANGULAR PROJECTION (BEAP)

The BEAP image representation is motivated by three properties. First, it represents large-FoV imagery without truncating the field of view. Second, it endows image tiles and their corresponding CSFs with an identical parameterization—a pair of θ and a pair of ϕ angles—which enables highly efficient PBF association and near-optimal GPU parallelization. Third, the induced ray sampling is more uniform in 3D space than that of conventional projective rendering, which empirically improves training efficiency and reconstruction quality.

Concretely, we sample rays uniformly in the spherical angular coordinates (θ, ϕ) (Eq. 6) and then map them into a discrete $w \times h$ image grid through an affine transformation (a scaling followed by a center shift):

$$(x, y) = \left\lfloor \frac{w\theta}{\text{FoV}_x} + \frac{w+1}{2}, \frac{h\phi}{\text{FoV}_y} + \frac{h+1}{2} \right\rfloor, \quad (11)$$

where $\text{FoV}_x, \text{FoV}_y$ are the camera's horizontal and vertical fields of view. As Fig. 4 shows, uniform sampling in (θ, ϕ) produces more balanced spatial coverage of the view frustum. This contrasts with projective sampling, which over-allocates samples to the periphery, and with equidistant projection, which oversamples near the image center.

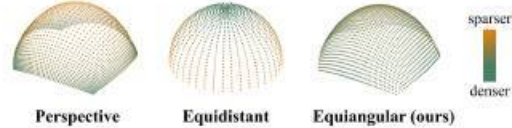


Figure 4: **Comparison of Ray Distributions under Varying Projections.** Pixels from three imaging spaces are back-projected onto the unit sphere to compare the induced ray distributions. Among the three, the proposed BEAP projection yields the most uniform angular coverage.

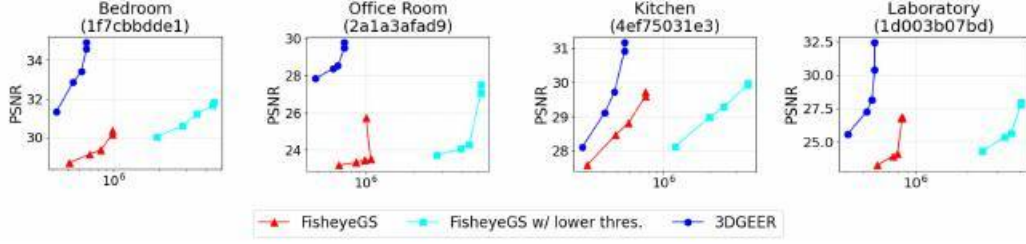


Figure 5: **PSNR $\uparrow$ as a Function of Gaussian Count.** Increasing the Gaussian count alone does not close the projective-exactness gap (full metrics in Fig. K.3 and Tab. K.3).



Figure 6: **Extreme-FoV Generalization.** 3DGEER reconstructs complete scenes using fewer Gaussians, whereas FisheyeGS exhibits severe quality degradation.

4 EXPERIMENTS

We evaluate 3DGEER on multiple datasets spanning both pinhole and fisheye camera models. Unless stated otherwise, all experiments use a single RTX 4090 GPU with 64 GB of RAM as the default configuration. The complete implementation details are deferred to Sec. G.

Datasets. To probe generic distorted cameras, the regime in which projective error is most consequential, we evaluate on all scenes of Fisheye ZipNeRF (Barron et al. 2023), five scenes of ScanNet++ (Yeshwanth et al. 2023), and two scenes of Aria (Lv et al. 2025). The ZipNeRF and ScanNet++ cameras have a 180° diagonal FoV, whereas Aria has a 110° circular FoV. For the pinhole regime we use Pinhole ZipNeRF and MipNeRF360 (Barron et al. 2021).

Evaluation Metrics and Protocol. We report reconstruction fidelity via PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$, efficiency via frame rate (FPS $\uparrow$), and model size via the Gaussian count $\downarrow$. To eliminate representation bias, every method’s output is reprojected to the native image space for full-FoV evaluation, and all comparisons are taken after 30k optimization iterations.

4.1 SCALING GAUSSIANS VS. PROJECTIVE EXACTNESS IN WIDE-FOV CAMERAS

Celarek et al. (2025) contend that increasing the Gaussian count (up to 1M) is sufficient to close the linear-approximation gap between splat-based and ray-based methods. This claim is substantiated only on small object-centric scenes and does not confront wide-FoV distortion. To test it directly, we scale the splat-based fisheye method FisheyeGS on ScanNet++ scenes by lowering its densification threshold, driving the model to 3.6–5M Gaussians (on an H200 GPU).

Fig. 5 reveals a persistent PSNR gap between FisheyeGS (red) and 3DGEER (blue) at matched Gaussian counts—measured from 4k to 30k iterations, beyond which densification typically saturates—which we attribute to the projective-exactness gap. The light-blue curve further shows that scaled-up FisheyeGS, notwithstanding its large increase in memory and compute, saturates at 29.3 dB PSNR, whereas 3DGEER reaches 32.1 dB with only 550–700K Gaussians. Full statistics appear in Tab. K.3

Table 2: **Comparison on the ScanNet++ Dataset.** Each method is trained on either the full FoV or the central region alone, then evaluated on the full, central, and peripheral regions to quantify robustness across distortion levels. Region-wise metrics are computed as pixel-mask-weighted averages; LPIPS is omitted because it is not separable by region (full statistics in Tab. K.2).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGS	Central	Persp.	-	-	-	31.26	0.948	-	-
FisheyeGS	Full	Eq.	27.81	0.946	0.139	32.44	0.956	23.28	0.914
EVER	Full	KB	29.47	0.924	0.167	29.93	0.924	28.72	0.925
3DGUT	Full	Eq.	30.64	0.944	0.150	31.87	0.945	28.84	0.937
3DGEER (Ours)	Central	BEAP	29.93	0.949	0.130	32.86	0.954	26.64	0.930
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Table 3: **Comparison on the ZipNeRF Dataset.** The protocol follows a cross-camera generalization setup: each method is trained on either Fisheye (FE, 1/8 resolution) or Pinhole (PH, 1/4 resolution) data and evaluated separately on both the Fisheye and Pinhole test sets (full statistics in Tab. K.9).

Train Data Test Data Method	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH	FE	PH
	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.18	26.44	0.858	0.868	0.211	0.239	19.43	26.61	0.791	0.889	0.247	0.212
EVER	25.17	26.14	0.880	0.851	0.153	0.237	22.90	26.45	0.835	0.862	0.207	0.222
3DGUT	24.77	25.59	0.879	0.804	0.183	0.324	18.61	25.96	0.662	0.841	0.300	0.270
3DGEER (Ours)	26.24	27.62	0.897	0.888	0.140	0.214	23.39	27.61	0.846	0.863	0.209	0.254

4.2 COMPREHENSIVE EVALUATION ON ALL-FOV NOVEL VIEW SYNTHESIS

We first benchmark 3DGEER against representative baselines on both fisheye and pinhole datasets, augmenting the standard comparison with analyses tailored to each dataset.

ScanNet++. Tab. 2 shows that 3DGEER dominates all baselines on every metric. To isolate distortion effects, we evaluate the central and peripheral regions separately (protocol in Sec. G). The baselines perform adequately at the center but degrade toward the periphery, whereas 3DGEER sustains high fidelity across the full field, producing sharper boundaries and fewer artifacts (Fig. K.4). Notably, even when trained solely on central crops, 3DGEER still exceeds FisheyeGS in all three regions.

We further assess extreme-FoV generalization (Fig. 6) by training at the native dataset FoV (180° diagonal) and testing at a substantially wider FoV (180°, circular). 3DGEER reconstructs complete scenes with fewer Gaussians, while splatting-based methods such as FisheyeGS degrade sharply at the periphery (additional extreme-FoV visualizations in Fig. K.1-K.2).

ZipNeRF. This dataset supplies frame-wise aligned pinhole and fisheye views. Beyond the standard fisheye evaluation, we conduct *cross-camera generalization*—training on fisheye and testing on pinhole, and conversely—to probe robustness. As Tab. 3 reports, 3DGEER is uniformly best across metrics, fully exploiting large-FoV training data through geometrically exact rendering. Trained on pinhole data, it still yields sizable PSNR gains on both test camera types, whereas FisheyeGS is competitive only in the pinhole-to-pinhole setting, and 3DGUT degrades markedly on fisheye after pinhole training. In aggregate, the geometrically exact methods EVER and 3DGEER generalize

Table 4: **Quantitative Results on the MipNeRF360 Dataset.** Alongside the quality metrics, FPS is measured on an RTX 4090 as the default benchmark. Values taken from the original works are annotated with $\dagger$ (measured on RTX 6000 Ada) and $\ddagger$ (measured on RTX 5090).

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	FPS $\uparrow$
3DGS	27.21	0.815	0.214	343
EVER	27.51	0.825	0.233	36
3DGRT	27.20	0.818	0.248	52 $\dagger$ / 68 $\ddagger$
3DGUT	27.26	0.810	0.218	265 $\dagger$ / 317 $\ddagger$
3DGEER (Ours)	27.76	0.821	0.210	327

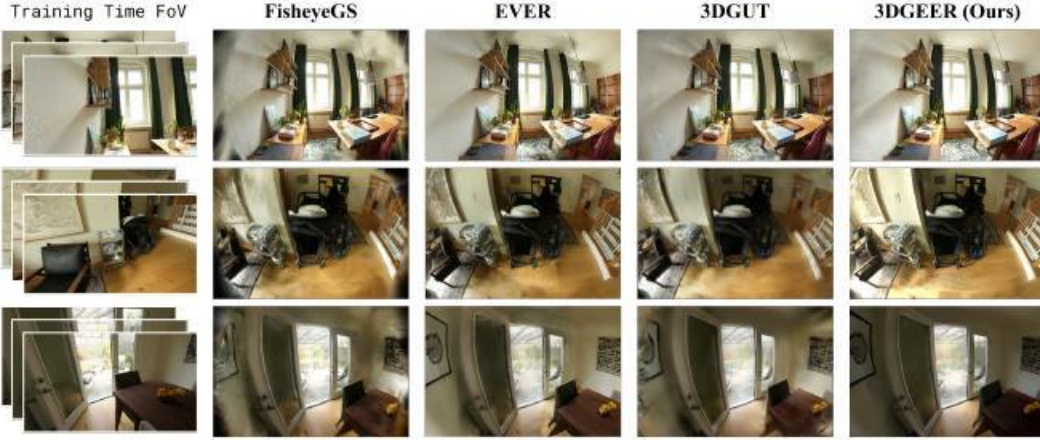


Figure 7: **Qualitative Results on the ZipNeRF Dataset with Cross-Camera Generalization.** The differences are most pronounced in the hardest Pinhole-(PH)-train / Fisheye-(FE)-test configuration, in which our method retains a substantial advantage.

Table 5: **Training-Time Transmittance Replacement on ScanNet++.** Substituting a splatting-based transmittance inflates the Gaussian count on account of its larger approximation error, yet a residual perceptual gap remains relative to our projective-exact formulation (full statistics in Tab. K.7).

Transmittance Method	Full FoV			Central		Peripheral		Avg. Gaussian Number (k) ↓
	PSNR ↑	SSIM ↑	LPIPS ↓	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑	
3DGS Splats	22.86	0.878	0.177	27.16	0.884	18.56	0.855	1396.4
FisheyeGS Splats	27.90	0.948	0.141	32.49	0.957	23.36	0.917	920.6
Projective-Exact (Ours)	31.50	0.953	0.126	32.64	0.955	28.94	0.945	591.8

best across camera models, and 3DGEER is superior in every case. Qualitatively (Fig. 7), 3DGEER recovers finer detail and is more resistant to peripheral artifacts in the most challenging pinhole-to-fisheye setting.

We additionally evaluate 3DGEER on the **MipNeRF360** dataset under standard narrow-FoV conditions. Per Tab. 4, 3DGEER establishes a new state of the art across all metrics. It runs more than $5\times$ faster than the exact ray-marching/tracing baselines EVER and 3DGRT and is the only projective-exact method whose runtime is comparable to 3DGS. It further surpasses the projective-approximation baselines (3DGS, 3DGUT) in PSNR, SSIM, and LPIPS. Further quantitative results on **Aria**—an egocentric fisheye dataset with a moderate 110° circular FoV and low-light imaging—together with additional visualizations, are reported in Sec. 7.

4.3 COMPREHENSIVE RUNTIME ANALYSIS

We analyze runtime on the RTX 4090 by comparing 3DGEER with splatting-based methods (3DGS on **MipNeRF360** and FisheyeGS on **ScanNet++**). Although the ray-based rendering stage of 3DGEER is marginally more expensive, the efficient ray-particle association afforded by PBF sharply reduces Gaussian duplication and sorting overhead (stage-wise timings in Tab. K.4), so the end-to-end throughput matches that of optimized splatting pipelines.

Quantitatively, PBF produces very tight bounds: each tile is associated with only ~ 475 Gaussians on average— $3\text{--}5\times$ fewer than EWA or UT (Tab. K.6)—which directly accounts for the $2.5\text{--}5\times$ speedup of the association stage (Tab. K.5). The projection-based schemes (EWA, UT), by contrast, conservatively overestimate the screen-space extent, incurring excessive overdraw and irregular memory access. Even with SnugBox (Hanson et al., 2025) acceleration, they remain limited by miscentered density approximations and frequently aggravate artifacts (Fig. 1). Beyond throughput, PBF preserves geometric accuracy and numerical stability and eliminates projective approximation error: as

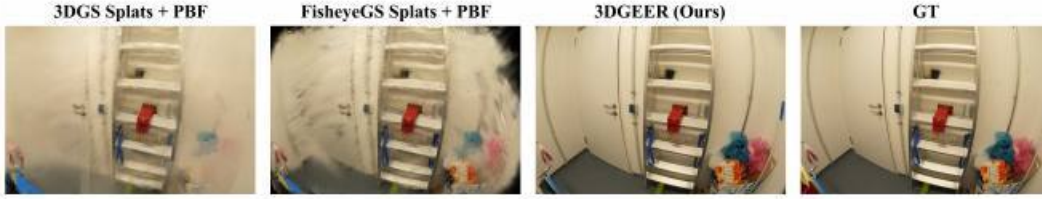


Figure 8: **Rendering-Time Transmittance-Function Replacement.** Substituting the splatting approximation for the projective-exact transmittance at test time alone induces mismatched culling.

Table 6: **BEAP vs. Alternative Supervision Schemes.** On ScanNet++, BEAP supervision surpasses the alternative projections on every metric (full statistics in Tab. K.8).

Method	Training FoV	Training Space	Full FoV			Central		Peripheral	
			PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	PSNR $\uparrow$	SSIM $\uparrow$
3DGEER w/o BEAP	Central	Persp.	29.84	0.943	0.131	32.21	0.951	26.23	0.915
3DGEER w/o BEAP	Full	Persp.	21.11	0.853	0.300	21.32	0.855	20.46	0.846
3DGEER w/o BEAP	Full	Eq.	31.05	0.948	0.135	32.21	0.952	28.56	0.935
3DGEER (Ours)	Full	BEAP	31.50	0.953	0.126	32.64	0.955	28.94	0.945

Tab. K.6 shows, PBF remains artifact-free even when ray-based rendering is applied to Gaussian fields trained under alternative association schemes.

4.4 LOCAL AFFINE APPROXIMATION VS. PROJECTIVE-EXACT INTEGRAL

To isolate the contribution of our projective-exact transmittance formulation (Eq. 4), we replace it with the standard local affine splat (the 2D conic used in 3DGS and FisheyeGS).

Rendering-time approximation. To sharpen the contrast between the two rendering formulations, we substitute the splatting-based approximation for the transmittance at test time only, holding all projective-exact components fixed during training. As Fig. 8 shows, coupling a splat-approximated transmittance with our exact projective association creates an inconsistency between the two stages that manifests as mismatched culling. This controlled setting magnifies and explains the grid-line artifact reported for 3DGUT (Fig. 1), where the inconsistency arises in the opposite direction: a ray-based transmittance paired with a conic-approximate association.

Training-time approximation. As reported in Tab. 5 on ScanNet++, when the splatting-based transmittance is also used during training while association remains projective-exact, the optimizer and densification partially absorb both the approximation error and the cross-view inconsistency introduced by splatting. The elevated loss produces stronger gradients throughout training, ultimately adding $1.6\text{--}2.4\times$ more Gaussians to fit the scene. This increase notwithstanding, the perceptual gap attributable to the approximation persists, indicating that densification alone cannot fully correct the error.

4.5 BEAP ON/OFF STUDY

We next compare BEAP supervision against widely used projection models on ScanNet++. In this ablation we hold the projective-exact integral and PBF association fixed and vary only the ray-sampling distribution used for supervision (pinhole, equidistant, and BEAP). Per Tab. 6 and Fig. K.5 BEAP supervision outperforms both alternatives on every metric. Its more uniform sampling yields smoother, more consistent gradients, which in turn improves convergence on fine detail.

The gap widens under pinhole cameras. At a fixed resolution, a well-balanced allocation of samples can matter more than a larger FoV per se. In the perspective domain the full FoV can even underperform the central region (Tab. 6 rows 1–2), because large-FoV images (Fig. K.2 Left) exhibit strongly non-uniform angular sampling: informative content concentrates at the center, while the low-information periphery occupies most of the image area and is thus disproportionately oversampled.

5 CONCLUSIONS

We presented 3DGEER, a complete closed-form solution for projective-exact volumetric Gaussian rendering derived from first principles. To make this formulation efficient, we introduced a closed-form PBF construction for ray-particle association that sustains high-throughput rendering while preserving exactness in projective geometry, and we proposed the BEAP image representation to provide effective color supervision under wide-FoV camera models. A comprehensive evaluation demonstrates that 3DGEER consistently outperforms state-of-the-art methods across multiple datasets, with the largest margins in the challenging wide-FoV regime.

ETHICS STATEMENT

This work relies exclusively on publicly available datasets (ScanNet++, ZipNeRF, Aria, and MipNeRF) and involves no human subjects, private data, or personally identifiable information. The methods advance 3D scene reconstruction and rendering, with prospective applications in XR, robotics, and autonomous driving. We foresee no direct harmful consequences; we note, however, that misuse of high-fidelity 3D reconstruction could raise privacy concerns. To mitigate this risk, we confine our experiments to research datasets in common use within the vision community.

REPRODUCIBILITY STATEMENT

We take substantial measures to ensure reproducibility. All mathematical derivations and proofs are given in the Appendix (Secs. B, C and D). Every dataset used in our experiments is publicly available (ScanNet++, ZipNeRF, Aria, and MipNeRF). The preprocessing pipeline, training protocol, and evaluation metrics are specified in Sec. G. Upon publication we will release our implementation together with training and evaluation scripts.

REFERENCES

- Jonathan T. Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P. Srinivasan. Mip-nerf: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE International Conference on Computer Vision (ICCV)*, 2021.
- Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Zip-nerf: Anti-aliased grid-based neural radiance fields. In *IEEE International Conference on Computer Vision, (ICCV)*, 2023.
- Adam Celarek, George Kopanas, George Drettakis, Michael Wimmer, and Bernhard Kerbl. Does 3d gaussian splatting need accurate volumetric rendering? In *Computer Graphics Forum (CGF)*, 2025.
- David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, pp. 19457–19467, 2024.
- Jorge Condor, Sébastien Speierer, Lukas Bode, Aljaz Bozic, Simon Green, Piotr Didyk, and Adrián Jarabo. Don’t splat your gaussians: Volumetric ray-traced primitives for modeling and rendering scattering and emissive media. *ACM Transactions on Graphics (TOG)*, 2025.
- Jonathan Courbon, Youcef Mezouar, Laurent Eckt, and Philippe Martinet. A generic fisheye camera model for robotic applications. In *IEEE/RSJ International Conference on Intelligent Robots and Systems, (IROS)*, pp. 1683–1688, 2007.
- Daniel Duckworth, Peter Hedman, Christian Reiser, Peter Zhizhin, Jean-François Thibert, Mario Lucic, Richard Szeliski, and Jonathan T. Barron. SMERF: streamable memory efficient radiance fields for real-time large-scene exploration. *ACM Transactions on Graphics (TOG)*, 2024.
- Zhongpai Gao, Benjamin Planche, Meng Zheng, Anwesa Choudhuri, Terrence Chen, and Ziyang Wu. 6dgs: Enhanced direction-aware gaussian splatting for volumetric rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs: Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10943–10952, 2025.
- Yuliang Guo, Sparsh Garg, S. Mahdi H. Miangoleh, Xinyu Huang, and Liu Ren. Depth any camera: Zero-shot metric depth estimation from any camera. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

-
- Florian Hahlbohm, Fabian Friederichs, Tim Weyrich, Linus Franke, Moritz Kappel, Susana Castillo, Marc Stamminger, Martin Eisemann, and Marcus Magnor. Efficient perspective-correct 3d gaussian splatting using hybrid transparency. In *Computer Graphics Forum (CGF)*, 2025.
- Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 21537–21546, 2025.
- Qiqi Hou, Randall Rauwendaal, Zifeng Li, Hoang Le, Farzad Farhadzadeh, Fatih Porikli, Alexei Bourd, and Amir Said. Sort-free gaussian splatting via weighted sum rendering. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2d gaussian splatting for geometrically accurate radiance fields. In Andres Burbano, Denis Zorin, and Wojciech Jarosz (eds.), *ACM SIGGRAPH Conference Papers*, 2024a.
- Letian Huang, Jiayang Bai, Jie Guo, and Yanwen Guo. Gs++: Error analyzing and optimal gaussian splatting. *CoRR*, 2024b.
- Juho Kannala and Sami S Brandt. A generic camera model and calibration method for conventional, wide-angle, and fish-eye lenses. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 2006.
- Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 2023.
- Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics (TOG)*, 43(4):1–15, 2024.
- Leonid Keselman and Martial Hebert. Approximate differentiable rendering with algebraic surfaces. In *European Conference on Computer Vision (ECCV)*, 2022.
- Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Yang-Che Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3d gaussian splatting as markov chain monte carlo. *Advances in Neural Information Processing Systems (NeurIPS)*, 37:80965–80986, 2024.
- Varun Ravi Kumar, Ciarán Eising, Christian Witt, and Senthil Kumar Yogamani. Surround-view fisheye camera perception for automated driving: Overview, survey & challenges. *IEEE Transactions on Intelligent Transportation Systems*, 24(4):3638–3659, 2023.
- Zimu Liao, Siyan Chen, Rong Fu, Yi Wang, Zhongling Su, Hao Luo, Li Ma, Linning Xu, Bo Dai, Hengjie Li, et al. Fisheye-gs: Lightweight and extensible gaussian splatting module for fisheye cameras. *ECCV Workshop*, 2024.
- Changkun Liu, Shuai Chen, Yash Sanjay Bhalgat, Siyan HU, Ming Cheng, Zirui Wang, Victor Adrian Prisacariu, and Tristan Braud. Gs-cpr: Efficient camera pose refinement via 3d gaussian splatting. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-gs: Structured 3d gaussians for view-adaptive rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 20654–20664, 2024.
- Zhaoyang Lv, Maurizio Monge, Ka Chen, Yufeng Zhu, Michael Goesele, Jakob Engel, Zhao Dong, and Richard Newcombe. Photoreal scene reconstruction from an egocentric device. In *ACM SIGGRAPH Conference Papers*, 2025.
- Alexander Mai, Peter Hedman, George Kopanas, Dor Verbin, David Futschik, Qiangeng Xu, Falko Kuester, Jonathan T Barron, and Yinda Zhang. EVER: exact volumetric ellipsoid rendering for real-time view synthesis. *IEEE International Conference on Computer Vision (ICCV)*, 2025.

- Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3dgs: High-quality radiance fields with limited resources. In *SIGGRAPH Asia Conference Papers*, 2024.
- Xiaoyang Mao, Lichan Hong, and Arie Kaufman. Splatting of curvilinear volumes. In *Proceedings Visualization '95*, pp. 61–68. IEEE, 1995.
- Christopher Mei and Patrick Rives. Single view point omnidirectional camera calibration from planar grids. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2007.
- Nicolas Moënné-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics (TOG)*, 2024.
- Hazem Rashed, Eslam Mohamed, Ganesh Sistu, Varun Ravi Kumar, Ciaran Eising, Ahmad El-Sallab, and Senthil Yogamani. Generalized object detection on fisheye cameras for autonomous driving: Dataset, representations and baseline. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, (WACV), pp. 2272–2280, 2021.
- Liu Ren, Hanspeter Pfister, and Matthias Zwicker. Object space ewa surface splatting: A hardware accelerated approach to high quality point rendering. In *Computer graphics forum (CGF)*, 2002.
- Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 347–362. Springer, 2024.
- Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- Ahmed Rida Sekkat, Yohan Dupuis, Varun Ravi Kumar, Hazem Rashed, Senthil Yogamani, Pascal Vasseur, and Paul Honeine. Synwoodscape: Synthetic surround-view fisheye camera dataset for autonomous driving. *IEEE Robotics and Automation Letters (RAL)*, 7(3):8502–8509, 2022.
- Chinmay Talegaonkar, Yash Belhe, Ravi Ramamoorthi, and Nicholas Antipa. Volumetrically consistent 3d gaussian rasterization. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pp. 10953–10963, 2025.
- Lee Westover. Footprint evaluation for volume rendering. In *Proceedings of the 17th annual conference on Computer graphics and interactive techniques*, pp. 367–376, 1990.
- Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moënné-Loccoz, and Zan Gojcic. 3dgt: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3d gaussian splatting for anti-aliased rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 20923–20931, 2024.
- Chandan Yeshwanth, Yueh-Cheng Liu, Matthias Nießner, and Angela Dai. Scannet++: A high-fidelity dataset of 3d indoor scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- Senthil Yogamani, Ciarán Hughes, Jonathan Horgan, Ganesh Sistu, Padraig Varley, Derek O’Dea, Michal Uricár, Stefan Milz, Martin Simon, Karl Amende, et al. Woodscape: A multi-task, multi-camera fisheye dataset for autonomous driving. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 9308–9318, 2019.
- Senthil Yogamani, David Unger, Venkatraman Narayanan, and Varun Ravi Kumar. Fisheyebevseg: Surround view fisheye cameras based bird’s-eye view segmentation for autonomous driving. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, (CVPR), pp. 1331–1334, 2024.
- Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, (CVPR), pp. 19447–19456, 2024a.

-
- Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (TOG)*, 2024b.
- Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-gs: Density control with pixel-aware gradient for 3d gaussian splatting. In *European Conference on Computer Vision (ECCV)*, pp. 326–342. Springer, 2024.
- Zichao Zhang, Henri Rebecq, Christian Forster, and Davide Scaramuzza. Benefit of large field-of-view cameras for visual odometry. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 801–808. IEEE, 2016.
- Hexu Zhao, Haoyang Weng, Daohan Lu, Ang Li, Jinyang Li, Aurojit Panda, and Saining Xie. On scaling up 3d gaussian splatting training. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pp. 29–538. IEEE, 2001.

CONTENTS

1 Introduction	1
2 Related Works	3
2.1 Splatting-Based Gaussian Rendering	3
2.2 Ray-Based Gaussian Rendering	3
3 Method	4
3.1 Rendering with Exact Projective Geometry	5
3.2 Ray-Particle Association with Exact Projective Geometry	5
3.3 Bipolar Equiangular Projection (BEAP)	6
4 Experiments	7
4.1 Scaling Gaussians vs. Projective Exactness in Wide-FoV Cameras	7
4.2 Comprehensive Evaluation on All-FoV Novel View Synthesis	8
4.3 Comprehensive Runtime Analysis	9
4.4 Local Affine Approximation vs. Projective-Exact Integral	10
4.5 BEAP On/Off Study	10
5 Conclusions	11
a Ethics Statement	12
b Reproducibility Statement	12
c LLM Usage Statement	18
A Appendix: Exactness and Limitation	18
B Appendix: Ray-Gaussian Transmittance with Projective Exactness	18
B.1 Preliminaries And Notations	18
B.2 Differentiable Rendering with Projective Exactness	19
B.3 Ray Integral of Isotropic 3D Gaussian	20
C Appendix: Cookbook of Gradient Computation	21
C.1 Chain Rule	21
C.2 Two-Stage Propagation	21
C.3 Gradients for Canonical Ray	21
C.4 Gradients for Whitening Matrix	22
C.5 Gradients for Scaling and Rotation	22
C.6 Gradients for 3D Mean	22
C.7 Numerical Stability of Our Differentiable Framework	22

864	D Appendix: Additional Details in Association	23
865	D.1 Particle Bounding Frustum (PBF) Solver	23
866	D.2 Details in Computing CSF-PBF Intersection	24
867	D.3 Relation to Prior Works	25
868		
869	E Appendix: BEAP Image Inverse Interpolation	25
870	Pinhole Camera Model	26
871	KB Fisheye Model	26
872		
873	F Appendix: High-level Algorithmic Summary	26
874		
875	G Appendix: Additional Implementation Details	27
876	Baselines	27
877	Implementation Details	27
878	Central and peripheral regions in ScanNet++ evaluation	28
879	Cross-camera generalization evaluation on ZipNeRF	28
880		
881	H Appendix: EWA Splatting Preliminaries	28
882	From camera coordinates to ray coordinates	28
883	Local affine approximation	28
884	Splatting approximation error	29
885		
886	I Appendix: Advantage over Prior Ray-Tracing Methods	29
887		
888	J Appendix: Results on Egocentric Dataset	29
889		
890	K Appendix: Additional Material and Results	31
891	K.1 Additional Material.	31
892	K.2 Additional Results.	31
893		
894		
895		
896		
897		
898		
899		
900		
901		
902		
903		
904		
905		
906		
907		
908		
909		
910		
911		
912		
913		
914		
915		
916		
917		

LLM USAGE STATEMENT

We employed a large language model (ChatGPT, OpenAI) solely as a writing assistant, specifically to (i) improve the clarity and readability of the manuscript, (ii) suggest stylistic revisions to figure captions and section transitions, and (iii) propose alternative phrasings of technical descriptions without altering their scientific content.

The model played no role in research ideation, algorithm design, data analysis, or experimental validation. All technical contributions, mathematical derivations, and results are the authors' own, and the authors assume full responsibility for the content of this paper.

A APPENDIX: EXACTNESS AND LIMITATION

We define our notion of exactness precisely: it denotes the *elimination of the linear approximation in projective geometry* in both the “rendering” and the “ray-particle association” stages, whose associated error does not diminish as the Gaussian count grows under our setting (see Sec. 4.1). We do not claim exactness in the full physical-volumetric sense; the framework still relies on approximations in depth ordering, self-attenuation, and inter-particle overlap, which constitute its limitations.

Limitations. Our formulation integrates each Gaussian independently and therefore does not explicitly resolve self-occlusion; depth ordering is instead handled by depth-based sorting. As the supplementary videos show, this can produce occasional popping artifacts. To our knowledge, EVER (Mai et al. 2025) is the sole method that resolves ordering and self-occlusion analytically, by approximating each Gaussian as a constant-density ellipsoid, but it does so at a substantial cost in rendering speed.

Notably, prior ray-tracing-based Gaussian renderers—Fuzzy Metaballs (Keselman & Hebert 2022) and its successors 3DGR/UT (Moënné-Loccoz et al. 2024; Wu et al. 2025)—adopt the same “maximum-response” assumption for ray-Gaussian interaction. Although this assumption formally disregards ordering and self-attenuation, its empirical error has been shown to be negligible (see Celarek et al. (2025), Fig. 9–11 / Sec. 7.1), independent of the Gaussian count. It follows that our depth-based ordering attains the same practical accuracy as existing ray-tracing-based Gaussian renderers.

Future work. Although the practical impact of ordering and self-occlusion is small in current Gaussian-based ray tracing, both factors fundamentally bound exact volumetric correctness. One promising direction is to incorporate analytic or learnable self-attenuation models—drawing on ellipsoidal integration (e.g., EVER (Mai et al. 2025)) while avoiding its runtime overhead. A second is to treat depth ordering more principledly, which may further suppress the rare popping artifacts seen under extreme configurations. We regard these limitations as orthogonal to our contribution on projective exactness; addressing them would render the framework more robust still for large-scale or high-dynamic-range scenes.

B APPENDIX: RAY-GAUSSIAN TRANSMITTANCE WITH PROJECTIVE EXACTNESS

B.1 PRELIMINARIES AND NOTATIONS

We adopt the classical parameterization of 3D Gaussian primitives from prior work (Zwicker et al. 2001). Each 3D Gaussian is specified by a mean $\mu \in \mathbb{R}^3$ and a 3D covariance Σ in the world coordinate system:

$$\mathcal{G}_{\Sigma, \mu}(\mathbf{x}) = \frac{1}{\rho |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^\top \Sigma^{-1} (\mathbf{x} - \mu) \right), \quad (\text{B.1})$$

where the normalization constant is set to $\rho = \sqrt{2\pi}$ and $\mathbf{x} \in \mathbb{R}^3$ is a world-space point. Unlike recent methods such as 3DGS (Kerbl et al. 2023) and 3DGUT (Wu et al. 2025), which drop the determinant factor $|\Sigma|^{1/2}$ to streamline splatting or association at render time, we retain it. The covariance admits the factorization $\Sigma = R S S^\top R^\top$, where the scaling matrix $S = \text{diag}(\mathbf{s})$ is built

from the scaling vector $\mathbf{s} \in \mathbb{R}^3$ and the rotation matrix $R \in \text{SO}(3)$ is obtained from a unit quaternion $\mathbf{q} \in \mathbb{R}^4$.

The transmittance contributed by a single 3D Gaussian particle along a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ is

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\Sigma, \mu}(\mathbf{o} + t\mathbf{d}) dt, \quad (\text{B.2})$$

where σ is the opacity coefficient of the Gaussian.

Finally, for a depth-sorted set of 3D Gaussian particles contributing to a given ray, the rendered color C is obtained by front-to-back compositing:

$$C(\mathbf{o}, \mathbf{d}) = \sum_i c_i(\mathbf{o}, \mu_i) T_i \prod_{j=1}^{i-1} (1 - T_j), \quad (\text{B.3})$$

where $c_i(\mathbf{o}, \mu_i)$ is the view-dependent color of the i -th Gaussian observed from the optical center $\mathbf{o}$, parameterized by spherical harmonics (SH) (Kerbl et al., 2023). Minimizing the reconstruction loss between the rendered and observed colors along each ray jointly optimizes the parameters $\mathbf{s}$, $\mathbf{q}$, μ , the coefficient σ , and the SH coefficients by backpropagation.

B.2 DIFFERENTIABLE RENDERING WITH PROJECTIVE EXACTNESS

The crux of projective-exact Gaussian rendering in a ray-marching formulation is a closed-form integral of Gaussian density and color along a 3D ray. We obtain it by transforming the ray—per Gaussian—into a canonical coordinate system in which the transmittance integral (Eq. B.2) is invariant and every Gaussian is isotropic, and then deriving the exact ray color from first principles.

Concretely, we introduce a canonical coordinate system shared by all 3D Gaussian primitives. A canonical point $\mathbf{u} \in \mathbb{R}^3$ is mapped to the world coordinates of a given Gaussian by:

$$\begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = H \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} = \begin{bmatrix} RS & \mu \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}, \quad (\text{B.4})$$

Under this transformation an infinitesimal canonical segment $\Delta \mathbf{u}$ maps to the world segment $\Delta \mathbf{x} = RS\Delta \mathbf{u}$.

To preserve each Gaussian’s transmittance contribution across coordinate systems, we require the elementary transmittance—the product of density and segment length—to be invariant under the transformation (see Fig. 2 Right):

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) |\Delta \mathbf{u}| = \mathcal{G}_{\Sigma, \mu}(\mathbf{x}) |\Delta \mathbf{x}|. \quad (\text{B.5})$$

This constraint absorbs the Jacobian determinant into the measure. Since $|\frac{\Delta \mathbf{x}}{\Delta \mathbf{u}}| = |RS| = |\Sigma|^{1/2}$, the canonical density takes the form:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2} \mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.6})$$

which is isotropic and shared by all primitives in canonical space.

Consequently, the exact closed-form transmittance integral of each Gaussian along $\mathbf{r}(t)$ is obtained by accumulating the isotropic canonical density along the transformed ray $\mathbf{r}_u(t) = \mathbf{o}_u + t\mathbf{d}_u$. The contribution of the segment $t < 0$ is negligible except when the optical center lies within the 3σ contour, a configuration that every 3D Gaussian rendering framework culls during optimization.

The integral admits a *closed form* in the perpendicular distance from the ray to the Gaussian center, as stated in Theorem 1

$$T(\mathbf{o}, \mathbf{d}) = \sigma \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{o}_u + t\mathbf{d}_u) dt = \sigma \exp\left(-\frac{1}{2} (D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.7})$$

where $D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d})$ is the perpendicular distance from the Gaussian center (the canonical-space origin) to the transformed ray $\mathbf{r}_u(t)$. Equivalently, this quantity is the minimal *Mahalanobis distance*

from any point on the ray to the Gaussian in world space, i.e., a measure of how closely the ray passes the particle distribution.

It remains to compute the perpendicular distance from the canonical origin to the transformed ray. We express the transformed ray's direction $\mathbf{d}_u$ and moment $\mathbf{m}_u$ as:

$$\mathbf{d}_u = (S^{-1}R^\top) \mathbf{d}, \quad \mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u, \quad (\text{B.8})$$

where $\mathbf{o}_u = (S^{-1}R^\top) (\mathbf{o} - \boldsymbol{\mu})$ is the optical center in canonical space. The squared distance $D_{\mu, \Sigma}^2(\mathbf{o}, \mathbf{d})$ is then computed efficiently as:

$$(D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2 = \frac{\mathbf{m}_u^\top \mathbf{m}_u}{\mathbf{d}_u^\top \mathbf{d}_u}. \quad (\text{B.9})$$

Substituting this squared distance into Eq. B.7 and the resulting transmittance into Eq. B.3 yields the accumulated color along the ray.

B.3 RAY INTEGRAL OF ISOTROPIC 3D GAUSSIAN

Theorem 1. *Given a ray and a standard isotropic 3D Gaussian, the ray-Gaussian integral has a closed-form proportional to:*

$$\exp\left(-\frac{1}{2}D^2\right),$$

where D is the perpendicular distance from the Gaussian center to the ray.

Proof. We first translate the coordinate system so that the Gaussian $\mathcal{G}$ is centered at the origin, translating the ray $\mathbf{r}_u$ accordingly, and then rotate the scene to align the ray direction with the z -axis. Because the integral is translation-invariant and the standard Gaussian is isotropic, neither transformation alters the value of the integral.

The translated standard isotropic 3D Gaussian is defined as:

$$\mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{u}) = \frac{1}{\rho} \exp\left(-\frac{1}{2}\mathbf{u}^\top \mathbf{u}\right), \quad (\text{B.10})$$

which matches Eq. B.6. Let the z -axis aligned ray as:

$$\mathbf{r}_u(t) = \begin{bmatrix} \mathbf{o}_u \\ 0 \end{bmatrix} + t \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \quad (\text{B.11})$$

the ray-Gaussian integral becomes:

$$\begin{aligned} \int_{t \in \mathbb{R}} \mathcal{G}_{\mathbf{I}, \mathbf{o}}(\mathbf{r}_u(t)) dt &= \frac{1}{\rho} \exp\left(-\frac{u_0^2 + v_0^2}{2}\right) \int_{t \in \mathbb{R}} \exp\left(-\frac{1}{2}t^2\right) dt \\ &= \frac{\sqrt{2\pi}}{\rho} \exp\left(-\frac{1}{2}D^2\right), \end{aligned} \quad (\text{B.12})$$

where $D^2 = \frac{u_0^2 + v_0^2}{2}$ denotes the squared perpendicular distance from the ray to the Gaussian center in canonical space. Thus, the integral is proportional to $\exp(-\frac{1}{2}D^2)$, as stated in theorem. $\square$

The canonical density shared by our 3D primitives (Eq. B.6) is identical in form to this isotropic case. For a world-space ray $(\mathbf{o}, \mathbf{d})$, the argument D depends additionally on the world-space Gaussian parameters $\boldsymbol{\mu}, \Sigma$, which define the transformation of the ray into canonical space. Hence, for each Gaussian contributing to the ray, the transmittance is:

$$T = \frac{\sqrt{2\pi}\sigma}{\rho} \exp\left(-\frac{1}{2}(D_{\mu, \Sigma}(\mathbf{o}, \mathbf{d}))^2\right), \quad (\text{B.13})$$

which recovers Eq. B.7 for the choice $\rho = \sqrt{2\pi}$.

C APPENDIX: COOKBOOK OF GRADIENT COMPUTATION

C.1 CHAIN RULE

Recall that the ray-Gaussian transmittance is $T = \sigma \alpha$ (Eq. B.7), with opacity coefficient σ and ray-Gaussian integral α . Let $\kappa = D_{\mu, \Sigma}^2$ denote the minimal squared Mahalanobis distance from the ray to the Gaussian, S the Gaussian scaling, and R the world-space rotation matrix. Additionally, given $\Sigma = RSS^\top R^\top$ as the 3D covariance matrix of the Gaussian in the world space, we have W_{pca} as the PCA whitening matrix satisfying:

$$W_{\text{pca}} = \Sigma^{-\frac{1}{2}} = S^{-1} R^\top, \quad \text{where } S^{-1} = \text{diag}\left(\frac{1}{\mathbf{s}}\right). \quad (\text{C.1})$$

As illustrated in Fig. 2 we can apply the chain rule to find derivatives *w.r.t.* the Gaussian parameters, e.g, scaling vector $\mathbf{s}$, rotation quaternion $\mathbf{q}$, and mean μ :

$$\frac{\partial T}{\partial \mathbf{s}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}, \quad (\text{C.2})$$

$$\frac{\partial T}{\partial \mathbf{q}} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}, \quad (\text{C.3})$$

$$\frac{\partial T}{\partial \mu} = \frac{\partial T}{\partial \kappa} \frac{\partial \kappa}{\partial \mu}. \quad (\text{C.4})$$

C.2 TWO-STAGE PROPAGATION

To reduce the overhead of shared propagation, we structure gradient computation as the two-stage scheme of 3DGS (Kerbl et al., 2023), comprising `RenderCUDA` (RC) and `PreprocessCUDA` (PC). RC performs ray-wise accumulation, accelerated by our Particle Bounding Frustum (PBF) association (Section 3.2); PC operates at Gaussian-wise complexity and exploits view-frustum culling for additional speedup.

From Eq. B.7 the gradient with respect to the opacity coefficient and the shared partial derivative for the Gaussian parameters in RC are:

$$\left. \frac{\partial T}{\partial \sigma} \right|_{\text{RC}} = \alpha, \quad \left. \frac{\partial T}{\partial \kappa} \right|_{\text{RC}} = -\frac{T}{2}. \quad (\text{C.5})$$

The remaining gradients with respect to the Gaussian parameters then propagate from the squared distance, i.e., $\frac{\partial \kappa}{\partial \mathbf{s}}, \frac{\partial \kappa}{\partial \mathbf{q}}, \frac{\partial \kappa}{\partial \mu}$.

C.3 GRADIENTS FOR CANONICAL RAY

We next backpropagate to the canonical ray parameters—the origin $\mathbf{o}_u$, direction $\mathbf{d}_u$, and moment $\mathbf{m}_u = \mathbf{o}_u \times \mathbf{d}_u$ defined in Eq. B.8. These quantities enter the Mahalanobis distance of Eq. B.9 directly and thus encode the canonical ray geometry.

$$\left. \frac{\partial \kappa}{\partial \mathbf{m}_u} \right|_{\text{RC}} = \left(\frac{2}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{m}_u, \quad (\text{C.6})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{o}_u} \right|_{\text{RC}} = \frac{\partial \kappa}{\partial \mathbf{m}_u} \times \mathbf{d}_u, \quad (\text{C.7})$$

$$\left. \frac{\partial \kappa}{\partial \mathbf{d}_u} \right|_{\text{RC}} = - \left(\frac{2\kappa}{\mathbf{d}_u^\top \mathbf{d}_u} \right) \mathbf{d}_u + \mathbf{o}_u \times \frac{\partial \kappa}{\partial \mathbf{m}_u}. \quad (\text{C.8})$$

C.4 GRADIENTS FOR WHITENING MATRIX

Recall that $\mathbf{d}_u = W_{\text{pca}} \mathbf{d}$ and $\mathbf{o}_u = W_{\text{pca}}(\mathbf{o} - \boldsymbol{\mu})$, where $\mathbf{o} = -R_c^\top \mathbf{t}_c$ denotes the world space optical center, and $\mathbf{d}$ denotes the ray direction. Given the gradients with respect to the ray parameters, we propagate them to the PCA whitening matrix W_{pca} through outer products with the world-space vectors:

$$\begin{aligned} \text{RenderCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} &= \frac{\partial \kappa}{\partial \mathbf{d}_u} \mathbf{d}^\top; \\ \text{PreprocessCUDA: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}} &= \frac{\partial \kappa}{\partial \mathbf{o}_u} (\mathbf{o} - \boldsymbol{\mu})^\top; \\ \text{Overall: } \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right| &= \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{RC}} + \left. \frac{\partial \kappa}{\partial W_{\text{pca}}} \right|_{\text{PC}}. \end{aligned} \quad (\text{C.9})$$

Here the gradient through $\mathbf{d}_u$ is accumulated ray-wise in `RenderCUDA`, whereas the gradient through $\mathbf{o}_u$ is computed Gaussian-wise in `PreprocessCUDA`; their sum constitutes the complete gradient with respect to the PCA transformation matrix W_{pca} .

C.5 GRADIENTS FOR SCALING AND ROTATION

Thus the scaling gradient $\frac{\partial \kappa}{\partial \mathbf{s}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{s}}$ is given by:

$$\left. \frac{\partial W_{\text{pca}}^{i,j}}{\partial s_k} \right|_{\text{PC}} = \begin{cases} -\frac{1}{s_k^2} R_{j,k} & \text{if } i = k \\ 0 & \text{otherwise} \end{cases}. \quad (\text{C.10})$$

For the rotation gradient $\frac{\partial \kappa}{\partial \mathbf{q}} = \frac{\partial \kappa}{\partial W_{\text{pca}}} \frac{\partial W_{\text{pca}}}{\partial \mathbf{q}}$, with the quaternion $\mathbf{q} = q_r + q_i \mathbf{i} + q_j \mathbf{j} + q_k \mathbf{k}$, the whitening matrix reads:

$$W_{\text{pca}} = 2 \begin{pmatrix} \frac{1}{s_1} \left(\frac{1}{2} - q_j^2 - q_k^2 \right) & \frac{1}{s_1} (q_i q_j + q_r q_k) & \frac{1}{s_1} (q_i q_k - q_r q_j) \\ \frac{1}{s_2} (q_i q_j - q_r q_k) & \frac{1}{s_2} \left(\frac{1}{2} - q_i^2 - q_k^2 \right) & \frac{1}{s_2} (q_j q_k + q_r q_i) \\ \frac{1}{s_3} (q_i q_k + q_r q_j) & \frac{1}{s_3} (q_j q_k - q_r q_i) & \frac{1}{s_3} \left(\frac{1}{2} - q_i^2 - q_j^2 \right) \end{pmatrix}. \quad (\text{C.11})$$

The resulting end gradients for the quaternion $\mathbf{q}$ are:

$$\begin{aligned} \left. \frac{\partial W_{\text{pca}}}{\partial q_r} \right|_{\text{PC}} &= 2 \begin{pmatrix} 0 & \frac{q_k}{s_1} & -\frac{q_j}{s_1} \\ -\frac{q_k}{s_2} & 0 & \frac{q_i}{s_2} \\ \frac{q_j}{s_3} & -\frac{q_i}{s_3} & 0 \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_i} \right|_{\text{PC}} = 2 \begin{pmatrix} 0 & \frac{q_j}{s_2} & \frac{q_k}{s_2} \\ \frac{q_k}{s_3} & -\frac{q_r}{s_3} & -\frac{2q_i}{s_3} \\ -\frac{2q_j}{s_3} & \frac{q_i}{s_3} & -\frac{q_r}{s_3} \end{pmatrix}, \\ \left. \frac{\partial W_{\text{pca}}}{\partial q_j} \right|_{\text{PC}} &= 2 \begin{pmatrix} -\frac{2q_j}{s_1} & \frac{q_i}{s_1} & -\frac{q_r}{s_1} \\ \frac{q_i}{s_2} & 0 & \frac{q_k}{s_2} \\ \frac{q_r}{s_3} & \frac{q_k}{s_3} & -\frac{2q_j}{s_3} \end{pmatrix}, \quad \left. \frac{\partial W_{\text{pca}}}{\partial q_k} \right|_{\text{PC}} = 2 \begin{pmatrix} -\frac{2q_k}{s_1} & \frac{q_r}{s_1} & \frac{q_i}{s_1} \\ -\frac{q_r}{s_2} & -\frac{2q_k}{s_2} & \frac{q_j}{s_2} \\ \frac{q_i}{s_3} & \frac{q_j}{s_3} & 0 \end{pmatrix}. \end{aligned} \quad (\text{C.12})$$

C.6 GRADIENTS FOR 3D MEAN

Finally, the end gradient with respect to the Gaussian 3D mean $\boldsymbol{\mu}$ is:

$$\left. \frac{\partial \kappa}{\partial \boldsymbol{\mu}} \right|_{\text{PC}} = -W_{\text{pca}}^\top \frac{\partial \kappa}{\partial \mathbf{o}_u}. \quad (\text{C.13})$$

C.7 NUMERICAL STABILITY OF OUR DIFFERENTIABLE FRAMEWORK

Numerical stability is governed jointly by the forward formulation, the backward gradient computation, and the choice of intermediate cached variables. Prior approaches such as GOF (Yu et al., 2024b) exhibit instability in backpropagation and consequently depend on auxiliary 3D-filtering heuristics to stabilize training. Our closed-form formulation with its exact gradient chain removes this instability entirely and thereby obviates any auxiliary filtering.



Figure C.1: **Artifacts without 3D filtering in GOF.** (Left) GOF without filtering of degenerate Gaussians; (Right) the stable result obtained under our forward and backward formulations.

The salient difference from prior pipelines is that they adopt an alternative expansion of $\mathbf{m}_u^\top \mathbf{m}_u$. In exact arithmetic the two forms are geometrically equivalent:

$$\mathbf{m}_u^\top \mathbf{m}_u = \|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2. \quad (\text{C.14})$$

When one axis of a 3D Gaussian is scaled close to zero, however, the canonical-space vectors $\mathbf{o}_u$ and $\mathbf{d}_u$ become nearly collinear: $\cos(\angle(\mathbf{o}_u, \mathbf{d}_u)) \rightarrow 1$ while $\|\mathbf{o}_u\|$ and $\|\mathbf{d}_u\|$ both diverge. The difference $\|\mathbf{o}_u\|^2 \|\mathbf{d}_u\|^2 - (\mathbf{o}_u^\top \mathbf{d}_u)^2$ is then a catastrophic cancellation of two large quantities, prone to overflow and, in practice, capable of returning negative values that destabilize backpropagation.

Fig. C.1 (Left) exhibits the resulting GOF artifacts: without auxiliary 3D filtering of degenerate Gaussians, severe artifacts emerge. The snowflake noise originates from the overflow in Eq. C.14, where negative values are returned prematurely and vanishingly small positive values inflate the Gaussian intensity. Because the EWA approximation with its Jacobian and conic is itself numerically stable, the artifacts are attributable chiefly to the inaccurate maximum-response computation, which additionally produces grid-line patterns.

By contrast, our CUDA rasterizer evaluates the cross product and its gradients directly (Eq. B.9 for the forward pass and Eqs. C.6-C.9 for the backward pass), which eliminates this numerical failure mode at its source: the forward pass is artifact-free and the backward pass yields accurate gradients. Fig. C.1 (Right) applies our forward and backward formulations to GOF and shows that, even without 3D filtering of degenerate (flat disk-like or needle-shaped) Gaussians, the instability-induced artifacts are removed.

D APPENDIX: ADDITIONAL DETAILS IN ASSOCIATION

Section D.1 gives the detailed derivation. Section D.2 presents the implementation considerations required for a stable PBF under large-FoV and omnidirectional cameras. Section D.3 clarifies the derivational relationships and states precisely the conditions under which prior formulas coincide with ours.

D.1 PARTICLE BOUNDING FRUSTUM (PBF) SOLVER

Following Eq. 8 let $P = (VH)^\top \in \mathbb{R}^{4 \times 4}$ be the plane transformation matrix from view space to canonical space. Taking the unknown θ bounds $c = \tan \theta_{1,2}$ as the representative case:

$$\mathbf{g}_u = -P_0 + cP_2 = \begin{bmatrix} -P_{00} + cP_{02} \\ -P_{10} + cP_{12} \\ -P_{20} + cP_{22} \\ -P_{30} + cP_{32} \end{bmatrix}, \quad (\text{D.1})$$

where P_i denotes the i -th row of the matrix, written as a 4×1 column vector, and P_{ij} its j -th element. Following Eq. 9 and bounding the 1-standard-deviation contour ellipsoid of the Gaussian ($\lambda = 1$), we obtain the quadratic equation:

$$((\mathbf{f} \circ P_2^\top) \cdot P_2) c^2 - 2((\mathbf{f} \circ P_0^\top) \cdot P_2) c + (\mathbf{f} \circ P_0^\top) \cdot P_0 = 0, \quad (\text{D.2})$$

where $\mathbf{f} = (1, 1, 1, -1)$ is a 1×4 row vector. We write this compactly as:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{02}c + \mathcal{T}_{00} = 0, \quad (\text{D.3})$$

where $\mathcal{T}$ is the 3×3 upper-triangular matrix with entries $\mathcal{T}_{ij} = (\mathbf{f} \circ \mathbf{P}_i^\top) \cdot \mathbf{P}_j$.

Similarly, for the bounds of ϕ , we have $c = \tan \phi_{1,2}$ satisfying:

$$\mathcal{T}_{22}c^2 - 2\mathcal{T}_{12}c + \mathcal{T}_{11} = 0. \quad (\text{D.4})$$

Notably, for a fixed camera-view configuration $[R_c \mid \mathbf{t}_c]$, the parameter matrix $\mathcal{T}$ can be computed efficiently from the Gaussian’s view-space covariance $\Sigma_c = R_c R S S^\top R^\top R_c^\top$ and mean $\mu_c = R_c \mu + \mathbf{t}_c$:

$$\mathcal{T}_{ij} = \lambda^2 \Sigma_c^{i,j} - (\mu_c \mu_c^\top)^{i,j}. \quad (\text{D.5})$$

The angular bounds, expressed through their tangent-space center and extent, then follow as:

$$\begin{aligned} \left(\frac{\tan \theta_1 + \tan \theta_2}{2}, \frac{\tan \phi_1 + \tan \phi_2}{2} \right) &= \left(\frac{\mathcal{T}_{02}}{\mathcal{T}_{22}}, \frac{\mathcal{T}_{12}}{\mathcal{T}_{22}} \right), \\ \|\tan \theta_1 - \tan \theta_2\| &= \frac{2\sqrt{\mathcal{T}_{02}^2 - \mathcal{T}_{22}\mathcal{T}_{00}}}{|\mathcal{T}_{22}|}, \\ \|\tan \phi_1 - \tan \phi_2\| &= \frac{2\sqrt{\mathcal{T}_{12}^2 - \mathcal{T}_{22}\mathcal{T}_{11}}}{|\mathcal{T}_{22}|}. \end{aligned} \quad (\text{D.6})$$

When the discriminant of either quadratic is negative—signaling that the optical center lies inside the Gaussian ellipsoid, so the angular bounds are undefined—we conservatively clamp the frustum angles to the camera’s maximum field of view (FoV).

This result is computed directly from the true 3D view-space covariance, unlike EWA and UT, which depend on intermediate conic approximations in screen space; the PBF formulation therefore delivers exactness and efficiency together.

D.2 DETAILS IN COMPUTING CSF-PBF INTERSECTION

We sample spherical angles uniformly within the maximum FoV. The resulting x, y, z coordinates—i.e., the ray directions $\mathbf{d}$ —are projected onto the unit sphere and then mapped to pixel space for inverse color interpolation (Sec. E). The sampled rays are grouped by their CSFs for subsequent Gaussian association.

For most pinhole cameras with a limited FoV, the PBF angular bounds lie within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. On this interval $\tan$ is strictly monotonic, so PBF-CSF intersections reduce to a direct comparison of the $\tan$ values of the respective bounds. In wide-FoV scenarios, however, the PBF angular bounds can fall outside the monotonic domain of $\tan$ even when the incidence angle of the Gaussian center remains within $[-\frac{\pi}{2}, \frac{\pi}{2}]$. A naive comparison of $\tan$ values then yields incorrect intersections, e.g., the spurious pairing $[\theta'_2, \theta_1]$ (Fig. D.1).

To recover the correct intersections, we apply a mirror transformation to the tangent values of both the PBF angular bounds and the CSF partitions. Following the CMEI camera model (Mei & Rives 2007), we shift the optical center by a

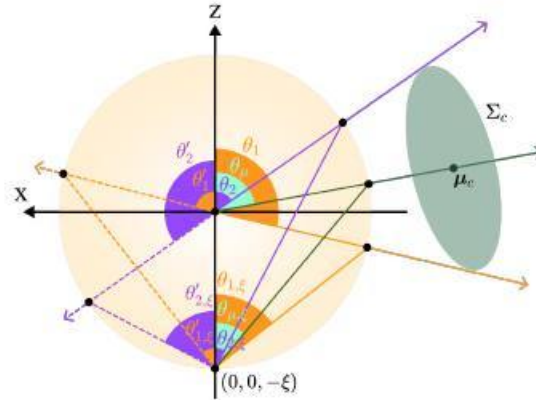


Figure D.1: Mirror Transformation of PBF Angular Bounds. A given pair of bounds generates four candidate mirrored angles $\theta_{1,\xi}, \theta'_{1,\xi}, \theta_{2,\xi}, \theta'_{2,\xi}$. For a Gaussian centered at μ_c , the two candidates nearest the Gaussian’s mirrored angle $\theta_{\mu,\xi}$ serve as the PBF bounds, yielding robust intersection with the mirror-transformed CSF partitions.

mirror parameter ξ , which induces the transformation:

$$\tan \theta_\xi = \frac{\tan \theta}{1 + \xi \frac{z}{\|z\|} \sqrt{1 + \tan^2 \theta}} = \frac{\sin \theta}{\cos \theta + \xi}, \quad (\text{D.7})$$

where θ_ξ is the mirror-transformed angle of θ on the xz -plane under mirror parameter ξ ; we fix $\xi = 1$ in all experiments.

Applied to the tangent pair $[\tan \theta_1, \tan \theta_2]$ returned by the PBF solver (Eq. D.6), this transformation produces four candidate mirror angles (Fig. D.1), whereas the Gaussian center maps to a single mirror-transformed angle $\theta_{\mu,\xi}$. We select the two candidates nearest $\theta_{\mu,\xi}$; these define the effective PBF bounds $[\theta_{1,\xi}, \theta_{2,\xi}]$, which are then compared with the CSF partitions to compute the correct intersections.

This construction guarantees robust, consistent angular comparisons even beyond the monotonic domain of $\tan$. As the timing analysis reports (Tab. K.5), the entire preprocessing pipeline—the mirror-angle PBF–CSF intersection and the canonical transformation of all Gaussians—completes in under 0.13 ms, confirming the efficiency of the approach even for thousands of Gaussians in wide-FoV scenes.

D.3 RELATION TO PRIOR WORKS

Substituting the perspective screen space coordinates:

$$u = \tan \theta, \quad v = \tan \phi$$

into our formulation recovers exactly the expressions of 2DGS (Huang et al., 2024a) and HTGS (Hahlbohm et al., 2025), which arise as the perspective specialization of our angular-domain formulation.

Concretely, 2DGS and HTGS begin from a screen-space or tangent-plane parameterization and therefore invoke assumptions valid only for perspective (pinhole) cameras—for instance, geometric identities that hold when rays are implicitly tied to a planar, z -parallel image plane. Such assumptions do not extend to fisheye or omnidirectional systems, for which the corresponding screen-space surface is curved. Our derivation instead proceeds from a camera-agnostic angular parameterization in 3D and directly parameterizes Camera Sub-Frustums (CSFs) and Particle Bounding Frustums (PBFs). These distinct starting assumptions alter the quantities and constraints available during derivation, and hence the conditions under which closed-form bounds and culling guarantees remain valid for generic cameras.

We emphasize that, although mathematics is a general tool, deriving these bounds under weaker assumptions—without recourse to screen-space structure—and validating them experimentally across both pinhole and wide-FoV camera families constitutes an essential part of our contribution.

The choice of parameterization also propagates to the downstream computational pipeline. Association, tiling/duplication, and sorting all depend on how Gaussians map to tiles or bins, and the angular (CSF-based) domain induces a different bucketing structure than pinhole screen-space tiling. This distinction is consequential for large-FoV scenes: under generic cameras a Gaussian’s bound may lie partially “behind” the camera in screen coordinates (Fig. D.1), so screen-space methods generate mirrored or otherwise incorrect tangent-based bounds (dashed line in Fig. D.1). To handle these cases robustly, we introduce the mirror parameters in the association step (Section D.2) and incorporate the Gaussian center into sorting and correction—components that a perspective-only derivation does not require.

E APPENDIX: BEAP IMAGE INVERSE INTERPOLATION

To construct training data, we sample rays that cover the original image FoV comprehensively and feed them to our CUDA renderer. Concurrently, we interpolate the ground-truth color at each input ray, providing accurate and uniform supervision during training.

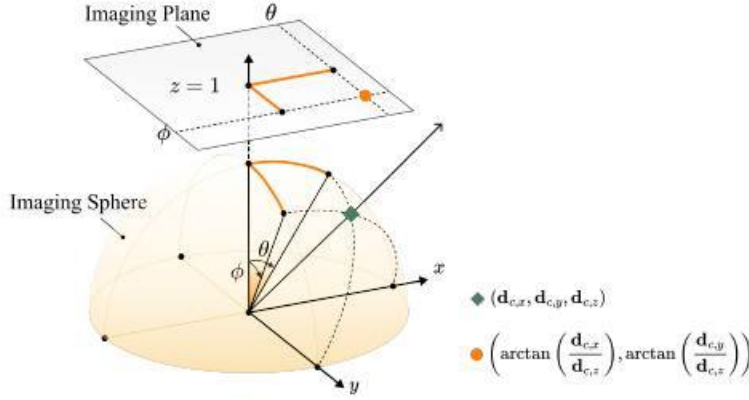


Figure E.1: **Spherical Angles Defining PBF and BEAP.** A view-space direction $\mathbf{d}_c$ is first normalized onto the optical sphere (green spade) and then projected angularly onto the xz and yz planes. The orange point marks the resulting coordinate (θ, ϕ) on the unfolded BEAP imaging plane.

Projecting the sampled rays onto the observed images begins by converting the incidence-angle projections θ and ϕ into normalized ray directions $\mathbf{d} = (x, y, z)^\top$:

$$x = \frac{\sin \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.1})$$

$$y = \frac{\cos \theta \sin \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}, \quad (\text{E.2})$$

$$z = \frac{\cos \theta \cos \phi}{\sqrt{\sin^2 \theta \cos^2 \phi + \cos^2 \theta \sin^2 \phi + \cos^2 \theta \cos^2 \phi}}. \quad (\text{E.3})$$

We support several camera models for the subsequent projection of the normalized rays $(x, y, z)^\top$ into pixel space $(x_p, y_p, 1)^\top$.

Pinhole Camera Model. For the pinhole model we have $\frac{x}{z} = \tan \theta$ and $\frac{y}{z} = \tan \phi$, so that:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.4})$$

KB Fisheye Model. The incidence angle φ is recovered from its projections:

$$r = \sqrt{\tan^2 \theta + \tan^2 \phi} \quad \text{and} \quad \varphi = \arctan(r). \quad (\text{E.5})$$

The fisheye-distorted angle is modeled by an even-order polynomial in φ :

$$\varphi_d = \varphi(1 + k_1\varphi^2 + k_2\varphi^4 + k_3\varphi^6 + k_4\varphi^8). \quad (\text{E.6})$$

The rays are then projected into pixel space, which supports inverse color interpolation via:

$$\begin{bmatrix} x_p \\ y_p \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{f_x \varphi_d}{r} & 0 & c_x \\ 0 & \frac{f_y \varphi_d}{r} & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \tan \theta \\ \tan \phi \\ 1 \end{bmatrix}. \quad (\text{E.7})$$

F APPENDIX: HIGH-LEVEL ALGORITHMIC SUMMARY

We summarize our volumetric rendering pipeline at a high level (Alg. 1 and Alg. 2). To expose the differences clearly, we mirror the structure of 3DGS Kerbl et al. (2023): newly introduced or replaced modules are highlighted in green, and substantially modified modules in yellow.

ALGORITHM 1: GPU Software Asso. & Render.

Function 3DGEER ($\mu, s, q, c, \sigma, r_c, V$) :

```
CullGaussian( $\mu, V$ ) ▷ Frustum Culling
 $F \leftarrow \text{InitCSFs}(w, h, K)$  ▷ Partition
 $\Sigma_c, \mu_c \leftarrow \text{ViewspaceGaussians}(s, q, V)$  ▷ Transform
 $F_{ID} \leftarrow \text{PBF\_Intersect}(\Sigma_c, \mu, F)$  ▷ Sub-frustum Asso.
 $W_{pca} \leftarrow \text{PCA\_Whitening}(s, q)$  ▷ Transform
 $\mathcal{G}_{ID}, F_K \leftarrow \text{DuplicateWithKeys}(W_{pca}, F_{ID})$  ▷ Indices (Gaussian) & Keys (FrustumID+Depth)
SortByKeys( $\mathcal{G}_{ID}, F_K$ ) ▷ Globally Depth Sort
 $R \leftarrow \text{IdentifyFrustumRanges}(F, F_K)$ 
 $I \leftarrow 0$  ▷ Init Canvas (w/ Rays)
for all CSFs  $f \leftarrow I$  do
  for all Rays  $i \leftarrow f$  do
     $r \leftarrow \text{GetFrustumRange}(R, f)$ 
     $m_u, d_u \leftarrow \text{Cano\_Ray}(W_{pca}, \mu, i)$  ▷ Transform
     $D_{\mu, \Sigma} \leftarrow \text{Dist}(m_u, d_u)$  ▷ Mah-distance
     $I[i] \leftarrow \text{BlendInOrder}(i, \mathcal{G}_{ID}, r, F_K, D_{\mu, \Sigma}, c, \sigma)$  ▷ Alpha Blending
  end
end
return  $I$ 
```

ALGORITHM 2: 3DGEER Initialization and Training Framework

 w, h : width and height of the training images K, V : intrinsics and extrinsic meta info.

```
 $\mu \leftarrow \text{SfM Points}$  ▷ Init Gaussian Attributes
 $s, q \leftarrow \text{InitAttributes}()$  ▷ Position
 $c, \sigma \leftarrow \text{InitCoefficients}()$  ▷ Scaling, Rotation
▷ Color, Opacity
while not converged do
   $r_c, \hat{C} \leftarrow \text{BEAP}(w, h, K)$  ▷ View Rays / Colors
   $C \leftarrow \text{3DGEER}(\mu, s, q, c, \sigma, r_c, V)$  ▷ Asso. & Render
   $\mathcal{L} \leftarrow \text{Loss}(C, \hat{C})$  ▷ See Alg. 1
   $\mu, s, q, c, \sigma \leftarrow \text{Adam}(\nabla \mathcal{L})$  ▷ Loss
  if IsRefinementIteration( $i$ ) then ▷ Backprop & Step
     $\mu, s, q, c, \sigma \leftarrow \text{Optimize}(\mu, s, q, c, \sigma)$  ▷ Kerbl et al. (2023)
  end
end
```

G APPENDIX: ADDITIONAL IMPLEMENTATION DETAILS

Baselines. We compare 3DGEER against leading volumetric particle rendering methods that target the projective approximation error of 3DGS (Kerbl et al. 2023). For large-FoV fisheye inputs, the baselines are FisheyeGS (Liao et al. 2024), EVER (Mai et al. 2025), and 3DGUT (Wu et al. 2025). In the pinhole regime, we additionally compare with 3DGS and 3DGRT (Moënné-Loccoz et al. 2024).

Implementation Details. For a fair comparison, we align our implementation with the original 3DGS in initialization and hyperparameters on ScanNet++ and MipNeRF360. On ZipNeRF we adopt the training configurations of EVER and SMERF (Duckworth et al. 2024) for all candidate methods. As is typical of ray-based frameworks, our method does not access gradients of mean2D for densification or pruning; instead it optimizes using gradients of the view-space mean3D. Unless noted otherwise, all models are trained under identical supervision strategies and architec-

tural settings. For the fisheye datasets (ScanNet++ and ZipNeRF), we precompute bijective grid mappings (Guo et al. 2025) and reproject every rendered output to the native KB-Fisheye image space (Kannala & Brandt 2006) prior to evaluation. ScanNet++ and MipNeRF360 experiments use full-resolution images; for ZipNeRF, whose high frame count and very large fisheye resolution are prohibitive, we use the highest resolution (Tab. 3) at which all baselines run reliably on a local desktop.

Central and peripheral regions in ScanNet++ evaluation. The rays sampled in the central part follows the range:

$$\theta \in [-\frac{w}{2f_x}, \frac{w}{2f_x}], \phi \in [-\frac{h}{2f_y}, \frac{h}{2f_y}].$$

The peripheral region comprises the rays sampled from the complementary angular range. Here w, h are the image width and height and f_x, f_y the focal lengths of the original KB camera model.

Cross-camera generalization evaluation on ZipNeRF. ZipNeRF provides two data sets: raw fisheye images and their undistorted counterparts, the latter obtained by distortion correction and central cropping. Although the two are aligned frame by frame, their camera poses and point clouds were reconstructed independently with COLMAP (Schönberger & Frahm 2016). Because COLMAP’s global optimization refines each trajectory independently, a single rigid transformation is insufficient to align the two pose sets and would introduce alignment error. To ensure reliable cross-camera evaluation free of structure-from-motion (SfM) error, we always use the same test-time camera poses as at training time; in the cross-camera experiments we vary only the intrinsic parameters and hold the extrinsics fixed. This yields fair comparisons uncontaminated by pose inconsistencies from preprocessing.

H APPENDIX: EWA SPLATTING PRELIMINARIES

From camera coordinates to ray coordinates. Adopting the coordinate conventions of EWA Splatting (Zwicker et al. 2001), the projective transformation for pinhole camera models maps camera- (view-) space points $\mathbf{t}$ to ray-space points $\mathbf{x}$ via $\tau : \mathbf{t} \rightarrow \mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix} = \tau(\mathbf{t}) = \begin{bmatrix} t_0/t_2 \\ t_1/t_2 \\ \|(t_0, t_1, t_2)^\top\| \end{bmatrix}, \quad (\text{H.1})$$

$$\mathbf{t} = \begin{bmatrix} t_0 \\ t_1 \\ t_2 \end{bmatrix} = \tau^{-1}(\mathbf{x}) = \begin{bmatrix} x_0/l \cdot x_2 \\ x_1/l \cdot x_2 \\ 1/l \cdot x_2 \end{bmatrix}, \quad (\text{H.2})$$

$$, \text{ where } l = \|(x_0, x_1, 1)^\top\|. \quad (\text{H.3})$$

Because these mappings are non-affine, evaluating the ray-space integral requires either computationally expensive procedures (Westover 1990; Mao et al. 1995) or local approximations (Zwicker et al. 2001; Kerbl et al. 2023).

Local affine approximation. Zwicker et al. (2001) introduced the local affine approximation τ_k so that the linear behavior is retained and the k -indexed reconstruction kernels remain Gaussians in 2D screen space. Defined by the first two terms of the Taylor expansion of τ at the camera-space point $\mathbf{t}_k$, it estimates the ray-space point $\mathbf{x}$ in a neighborhood of $\mathbf{x}_k$ by:

$$\mathbf{x} = \tau(\mathbf{t}) = \mathbf{x}_k + \mathbf{J}_\tau^{\mathbf{t}_k}(\mathbf{t} - \mathbf{t}_k), \quad (\text{H.4})$$

where $\mathbf{J}_\tau^{\mathbf{t}_k}$ is the Jacobian of τ evaluated at $\mathbf{t}_k$. Under this EWA local affine transformation, the k -indexed 3D Gaussian $\mathcal{G}_{\Sigma, \mu}$ in a given camera coordinate maps to an approximating 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}$ in ray coordinates. For $\forall \mathbf{x}$ around $\mathbf{x}_k$, we have:

$$\mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x}) = \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mathbf{x}_k)^\top \Sigma_k^{-1}(\mathbf{x} - \mathbf{x}_k)\right), \quad (\text{H.5})$$

$$\text{where } \Sigma_k = \mathbf{J}_\tau^{\mathbf{t}_k} \Sigma (\mathbf{J}_\tau^{\mathbf{t}_k})^\top. \quad (\text{H.6})$$



Figure I.1: **Qualitative Results on Stump (Mip-NeRF360).** On the novel-view-synthesis benchmark (Left, MipNeRF360, pinhole), 3DGEER outperforms both 3DGS-based (Gaussian Splatting) and 3DPRT-based (Particle Ray Tracing) methods while retaining high runtime efficiency. The largest differences are highlighted in the colored boxes.

Integrating the 3D Gaussian $\mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}$ along one coordinate axis yields a 2D Gaussian $\mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}$:

$$\int_{\mathbb{R}} \mathcal{G}_{\Sigma_k, \mathbf{x}_k}^{(3)}(\mathbf{x}) dx_2 = \mathcal{G}_{\hat{\Sigma}_k, \hat{\mathbf{x}}_k}^{(2)}(\hat{\mathbf{x}}), \quad (\text{H.7})$$

where $\hat{\mathbf{x}} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$ and the mean is $\hat{\mathbf{x}}_k = \begin{bmatrix} x_{k,0} \\ x_{k,1} \end{bmatrix}$; the 2×2 covariance $\hat{\Sigma}_k$ is obtained from the 3×3 matrix Σ_k by deleting its third row and column.

Splatting approximation error. Although the ray-space density is no longer Gaussian once a general projective transformation is applied to the view space, its exact form follows by substitution via Eq. H.2

$$\begin{aligned} \mathcal{G}_{\Sigma, \mu}(\mathbf{t}) &= \mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) \\ &= \frac{1}{(2\pi)^{\frac{3}{2}} |\Sigma_k|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k)^{\top} \Sigma_k^{-1} (\tau^{-1}(\mathbf{x}) - \mathbf{t}_k) \right), \end{aligned} \quad (\text{H.8})$$

Accordingly, the approximation error is quantified by Monte Carlo integration of:

$$\mathcal{E}_k = \int_{\mathbb{R}} |\mathcal{G}_{\Sigma, \mu}(\tau^{-1}(\mathbf{x})) - \mathcal{G}_{\Sigma_k, \mathbf{x}_k}(\mathbf{x})| dx_2. \quad (\text{H.9})$$

Here x_2 denotes the coordinate along the ray direction in ray space.

I APPENDIX: ADVANTAGE OVER PRIOR RAY-TRACING METHODS

Pure ray tracing (e.g., 3DGRT) is in principle projective-exact, but in practice the BVH that 3DGRT uses for intersection tests employs an inscribed proxy icosahedron for speed; this proxy is tight at the vertices, yet its face centers lie $\sim 20\%$ inward by construction. Such proxy shapes reduce intersection accuracy, corrupt the accumulated gradients that drive density control, and ultimately produce incorrect geometry (e.g., the missing leaves in Fig. I.1). Moreover, intersection tests for distant, small, and cluttered Gaussians are limited by floating-point precision, which yields noisy renderings (e.g., in Fig. 15 of the 3DGRT paper, the leftmost block’s native rendering shows visible speckles in the tree and background). Even 3DGUT with its UT approximation outperforms 3DGRT on MipNeRF360. As a controlled comparison, we render from the same pretrained MipNeRF360-Stump model using 3DGRT’s ray-tracing engine and our 3DGEER pipeline, reporting PSNR $\uparrow$ / SSIM $\uparrow$ / LPIPS $\downarrow$: 3DGRT, 25.79 / 0.745 / 0.264; ours, 26.83 / 0.774 / 0.246 (a gain of +1.04 / +0.029 / -0.018). 3DGEER constructs a projective-exact and efficient pipeline and additionally optimizes under BEAP color supervision, attaining both higher throughput and higher accuracy (Tab. 4).

J APPENDIX: RESULTS ON EGOCENTRIC DATASET

Contemporary egocentric devices typically employ wide FoVs, yet existing pipelines struggle to preserve reconstruction quality at full resolution or under cropped views. The difficulty is compounded



Figure J.1: **Qualitative Results on the Aria Dataset.** Our method recovers sharper floor textures in the indoor scene and clearer distant-tree detail in the outdoor scene.

Table J.1: **Quantitative Comparison on Aria.** Higher PSNR and SSIM and lower LPIPS indicate better quality.

Method	Scene		PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGEER (Ours)	livingroom	(indoor)	36.398	0.933	0.226
	steakhouse	(outdoor)	30.662	0.881	0.246
FisheyeGS	livingroom	(indoor)	35.686	0.927	0.231
	steakhouse	(outdoor)	29.989	0.869	0.261

by low-light imaging: constrained light intake, particularly with small sensors and aperture-limited lenses, amplifies degradation.

We evaluate on the Aria egocentric dataset ($\sim 110^\circ$ circular FoV). Our method attains an average PSNR of 33.5 dB, exceeding FisheyeGS (32.8 dB) on every metric (Tab. J.1). Qualitatively, our reconstructions retain sharper peripheral detail in the indoor scene (Fig. J.1 Left) and recover distant structures such as trees in the outdoor scene (Fig. J.1 Right). The +0.7 dB gain falls between those observed on pinhole datasets (MipNeRF360: +0.3 dB; ZipNeRF cross-camera: +2.0 dB) and on strongly distorted wide-FoV datasets (ScanNet++/ZipNeRF, 180° diagonal FoV: +0.9–4.8 dB), consistent with Aria’s moderate FoV and distortion level.

K APPENDIX: ADDITIONAL MATERIAL AND RESULTS

K.1 ADDITIONAL MATERIAL.

Code, video, and an HTML page are provided in the Supplementary Material.

K.2 ADDITIONAL RESULTS.



Figure K.1: **Out-of-Distribution (OOD) Region in Fig. 6** 3DGS collapses to black owing to the singularity $\tan 90^\circ \rightarrow \infty$, whereas our method renders views up to 180° robustly and reconstructs the green chair where splat-based methods (e.g., FisheyeGS) fail. Analogously, in the zoom-in of Fig. 6 our method preserves structures such as the corridor and fridge, the table and door, and the window and cabinet.



Figure K.2: **Extreme-FoV Generalization.** (Left) Our method versus 3DGS on extreme-FoV pinhole (170°) views, with both trained on undistorted central regions. (Right) Zoom-out of Fig. 6 at the full circular 180° FoV, where our method generalizes robustly to out-of-distribution regions.

Table K.1: **Per-Sequence Quantitative Results on MipNeRF360.** Aggregate results appear in Tab. 4

Method	Metric	Bicycle	Bonsai	Counter	Garden	Kitchen	Stump	Flowers	Room	Treehill
Ours	PSNR $\uparrow$	25.63	32.23	29.05	27.60	31.78	26.83	21.83	32.09	22.83
	SSIM $\uparrow$	0.774	0.946	0.910	0.860	0.931	0.774	0.613	0.946	0.637
	LPIPS $\downarrow$	0.223	0.175	0.195	0.136	0.120	0.246	0.340	0.108	0.344

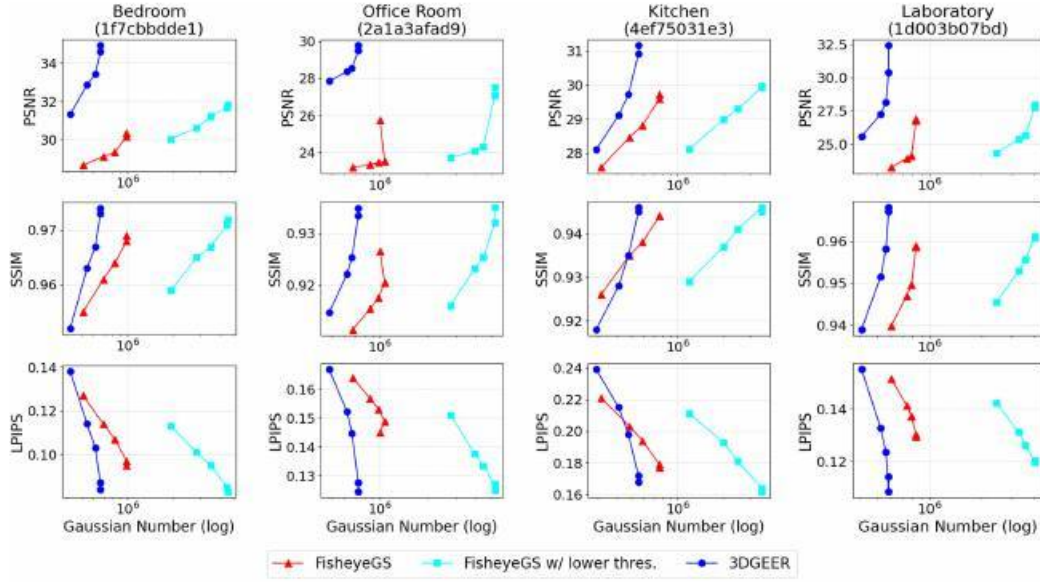


Figure K.3: $\text{PSNR}\uparrow$, $\text{SSIM}\uparrow$, and $\text{LPIPS}\downarrow$ as a Function of Gaussian Count. Increasing the Gaussian count alone does not close the projective-exactness gap.

Table K.2: Per-Sequence Quantitative Results on ScanNet++ (Full FoV). Aggregate results appear in Tab. 2

Scene	EVER			FisheyeGS			3DGUT			3DGEER (Ours)		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.24	0.916	0.164	24.74	0.932	0.146	29.26	0.934	0.147	29.18	0.940	0.141
Office Room	27.80	0.899	0.169	25.75	0.927	0.145	29.37	0.928	0.145	29.78	0.935	0.125
Bedroom	32.60	0.948	0.131	31.95	0.970	0.095	34.65	0.967	0.112	34.94	0.974	0.084
Kitchen	29.77	0.924	0.213	29.72	0.944	0.177	30.78	0.941	0.196	31.17	0.946	0.168
Laboratory	28.96	0.936	0.156	26.91	0.959	0.129	29.12	0.948	0.151	32.43	0.968	0.114

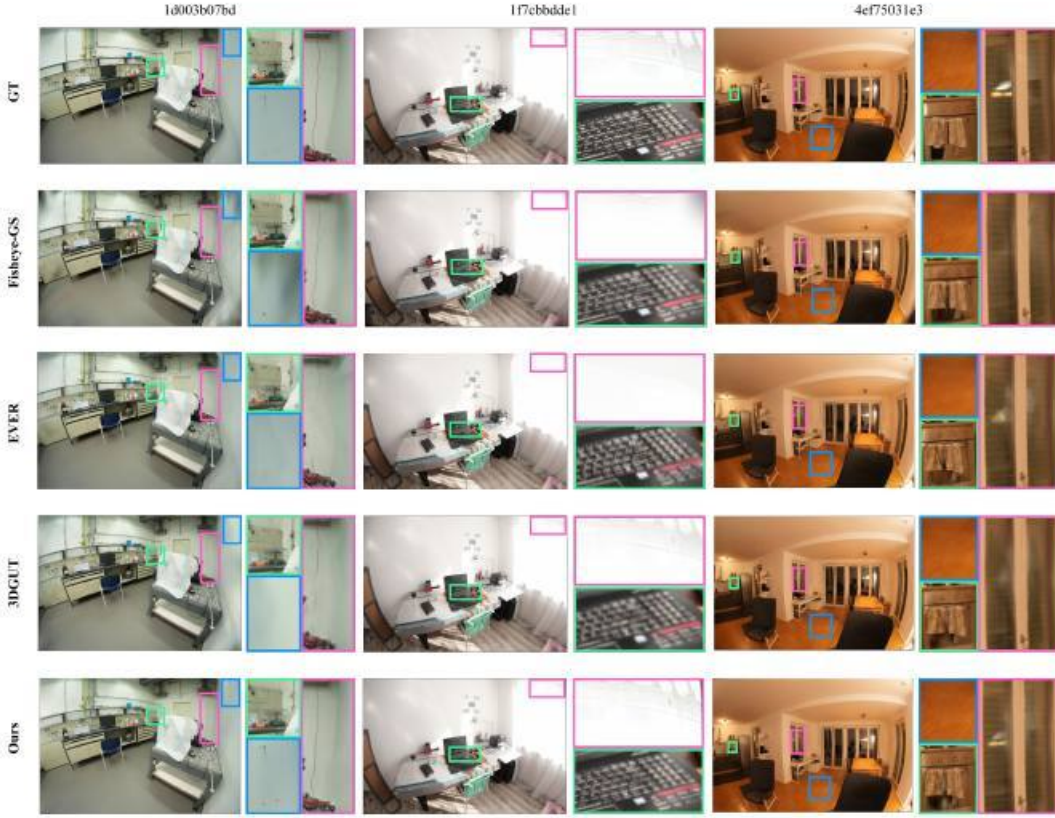


Figure K.4: **Qualitative Results on the ScanNet++ Dataset.** The largest differences appear in the zoomed-in colored boxes, notably in fine structure and artifact-prone regions. Relative to the baselines, our method yields superior peripheral reconstruction over FisheyeGS, fewer artifacts and sharper detail than EVER, and crisper object boundaries and textures—wires, keyboards, and wooden floors—than 3DGUT.

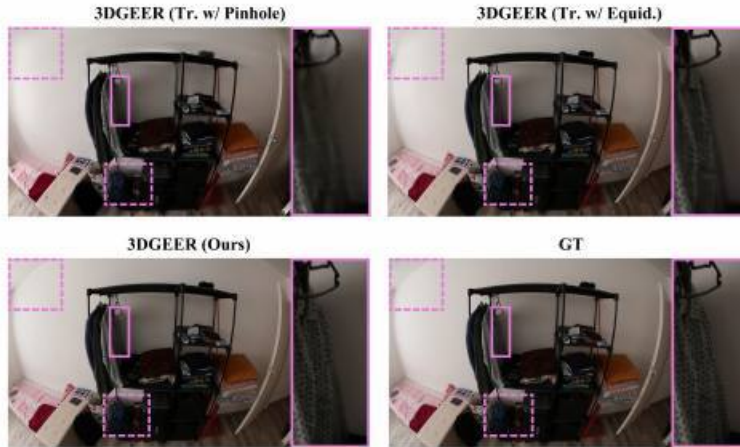


Figure K.5: **Impact of BEAP Supervision.** On ScanNet++, the balanced angular sampling of BEAP yields smoother gradients and better preserves geometric and textural detail than pinhole or equidistant sampling (full quantitative statistics in Tab. [K.8](#)).

Table K.3: **Quantitative Results on ScanNet++**. PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$ as a function of Gaussian count, reported for Bedroom, Office Room, Kitchen, and Laboratory in that order.

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	28.67	0.955	0.127	512K	30.03	0.959	0.113	1914K	31.33	0.952	0.138	422K
7k	29.12	0.961	0.114	695K	30.61	0.965	0.101	2829K	32.87	0.963	0.114	543K
10k	29.36	0.964	0.107	827K	31.21	0.967	0.095	3489K	33.42	0.967	0.103	616K
20k	30.17	0.968	0.097	985K	31.68	0.971	0.085	4467K	34.59	0.973	0.087	664K
30k	30.36	0.969	0.095	985K	31.84	0.972	0.083	4535K	34.94	0.974	0.084	664K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.19	0.911	0.164	677K	23.72	0.916	0.151	2778K	27.84	0.915	0.167	480K
7k	23.33	0.916	0.157	867K	24.06	0.923	0.138	3936K	28.37	0.922	0.152	621K
10k	23.45	0.918	0.153	983K	24.31	0.925	0.133	4481K	28.54	0.925	0.145	668K
20k	23.51	0.921	0.149	1072K	27.07	0.932	0.127	5272K	29.49	0.933	0.128	732K
30k	25.75	0.927	0.145	1001K	27.50	0.935	0.125	5272K	29.78	0.935	0.125	732K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	27.58	0.926	0.221	314K	28.12	0.929	0.211	1191K	28.11	0.918	0.239	292K
7k	28.47	0.935	0.203	485K	28.99	0.937	0.193	1997K	29.11	0.928	0.215	410K
10k	28.82	0.938	0.194	588K	29.29	0.941	0.181	2491K	29.73	0.935	0.198	476K
20k	29.60	0.944	0.179	760K	29.93	0.946	0.164	3563K	30.91	0.945	0.172	552K
30k	29.72	0.944	0.177	760K	29.97	0.945	0.162	3563K	31.17	0.946	0.168	552K

Iter	FisheyeGS				FisheyeGS w/ Low Thres.				3DGEER			
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Gau $\downarrow$
4k	23.27	0.940	0.152	602K	24.32	0.945	0.142	2429K	25.59	0.939	0.155	406K
7k	23.94	0.947	0.141	740K	25.36	0.953	0.131	3258K	27.27	0.952	0.133	524K
10k	24.12	0.950	0.137	788K	25.65	0.956	0.126	3569K	28.14	0.958	0.124	562K
20k	26.76	0.959	0.130	838K	27.77	0.961	0.120	4035K	30.39	0.967	0.108	582K
30k	26.91	0.959	0.129	838K	27.97	0.961	0.120	4035K	32.43	0.968	0.114	582K

Table K.4: **Runtime Analysis vs. Other Methods**. 3DGEER is compared with efficient splatting-based methods on MipNeRF360 and ScanNet++, with a thorough comparison in stage-wise and total runtime.

Dataset	Method	Avg. Timings (ms) $\downarrow$					
		Prep.	Dup.	Sort	Render	Total	
MipNeRF360	3DGS	0.32	0.27	0.65	1.40	2.92	
	3DGEER (Ours)	0.37	0.17	0.27	2.10	3.06	
ScanNet++	FisheyeGS	0.10	0.63	1.45	2.33	4.70	
	3DGEER (Ours)	0.13	0.26	0.59	2.89	3.98	

Table K.5: **Runtime Analysis: Asso. Method on ScanNet++**. The efficiency of the PBF-based association is compared to the splatting-based EWA and UT with acceleration option SnugBox integrated into 3DGEER pipeline.

3DGEER Asso.	Prep.	Avg. Timings (ms) ↓				Total	Avg. FPS ↑
		Dup.	Sort	Render			
EWA	0.13	1.17	3.27	12.69		17.84	56
+ SnugBox	0.13	0.60	1.42	6.22		8.68	115
UT	0.13	0.71	1.94	7.97		11.16	90
+ SnugBox	0.13	0.28	0.64	3.16		4.38	228
PBF (Ours)	0.13	0.26	0.59	2.89		3.98	251

Table K.6: **Impact of PBF-CSF Association**. The evaluation employs the artifact-sensitive LPIPS metric. To assess cross-model robustness, each variant is also tested using models trained with the other association strategies. Additionally, we report the memory cost during rendering along with the average number of Gaussian association per Tile, and the performance gap Δ LPIPS between in-method and cross-method evaluations to quantify consistency and generalization.

Asso. Method	Gaus. ↓ per Tile		Mem. ↓ (GB)	LPIPS ↓	Cross-LPIPS ↓	Δ LPIPS ↓ (1e-2)
	mean	std				
EWA	2203	1447	2.2	0.1250	0.1321	0.71
UT	1377	1058	1.4	0.1249	0.1302	0.53
PBF (Ours)	475	427	0.63	0.1245	0.1251	0.06

Table K.7: **Per-Scene Training-Time Transmittance-Function Replacement**. Aggregate results appear in Tab. 5.

Scene	3DGS Splats + PBF				FisheyeGS Splats + PBF				3DGEER (Ours)			
	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)	PSNR	SSIM	LPIPS	Gaus. Num (k)
Storage	19.27	0.859	0.173	1293	24.75	0.933	0.150	781	29.18	0.940	0.141	429
Office Room	20.89	0.848	0.182	1527	25.89	0.928	0.150	1119	29.78	0.935	0.125	732
Bedroom	27.39	0.923	0.119	1542	31.96	0.970	0.096	1079	34.94	0.974	0.084	664
Kitchen	25.15	0.890	0.239	1241	29.87	0.949	0.182	801	31.17	0.946	0.168	552
Laboratory	21.60	0.869	0.173	1379	27.03	0.962	0.131	823	32.43	0.968	0.114	582

Table K.8: **Per-Scene Effect of the Supervision Space**. Aggregate results appear in Tab. 6.

Supervision Space Scene	3DGEER w/o BEAP Central Perspective			3DGEER w/o BEAP Perspective			3DGEER w/o BEAP Equidistant			3DGEER (Ours) BEAP		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
Storage	28.42	0.931	0.143	19.74	0.834	0.302	29.03	0.938	0.142	29.18	0.940	0.141
Office Room	28.90	0.927	0.133	21.12	0.825	0.291	29.71	0.932	0.126	29.78	0.935	0.125
Bedroom	33.86	0.965	0.090	24.38	0.882	0.240	34.60	0.973	0.096	34.94	0.974	0.084
Kitchen	30.65	0.941	0.172	23.23	0.889	0.338	31.23	0.941	0.177	31.17	0.946	0.168
Laboratory	27.35	0.951	0.118	17.08	0.834	0.329	30.65	0.956	0.135	32.43	0.968	0.114

Table K.9: **Per-Sequence Quantitative Results on ZipNeRF**. Sequences are ordered Alameda, Berlin, London, NYC; aggregate results appear in Tab. 3

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	20.74	23.64	0.820	0.802	0.223	0.277	17.39	23.74	0.736	0.839	0.265	0.240
EVER	22.46	23.06	0.852	0.777	0.144	0.274	20.06	22.83	0.789	0.794	0.213	0.250
3DGUT	21.78	22.39	0.841	0.718	0.193	0.368	17.03	22.58	0.588	0.765	0.329	0.311
Ours	24.60	25.00	0.857	0.839	0.154	0.241	21.49	25.21	0.792	0.807	0.232	0.277

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.25	28.25	0.895	0.924	0.206	0.209	20.08	27.86	0.848	0.929	0.233	0.194
EVER	25.26	26.97	0.912	0.907	0.155	0.221	23.18	27.72	0.877	0.913	0.196	0.212
3DGUT	25.37	27.02	0.904	0.876	0.197	0.288	18.57	27.43	0.730	0.898	0.297	0.249
Ours	25.51	29.15	0.916	0.925	0.166	0.194	23.58	29.51	0.878	0.915	0.219	0.228

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	23.57	26.65	0.829	0.861	0.253	0.254	20.83	27.33	0.768	0.883	0.288	0.229
EVER	24.59	26.83	0.823	0.859	0.218	0.223	23.14	26.98	0.793	0.860	0.268	0.235
3DGUT	25.61	26.25	0.854	0.803	0.221	0.336	19.82	26.41	0.669	0.833	0.313	0.291
Ours	26.60	27.84	0.895	0.882	0.142	0.230	23.95	27.15	0.840	0.846	0.217	0.289

Train Data	1/8 FE		1/8 FE		1/8 FE		1/4 PH		1/4 PH		1/4 PH	
Test Data	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH	1/8 FE	1/4 PH
Method	PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$		PSNR $\uparrow$		SSIM $\uparrow$		LPIPS $\downarrow$	
FisheyeGS	24.15	27.21	0.890	0.885	0.162	0.215	19.41	27.50	0.811	0.907	0.203	0.183
EVER	28.39	27.71	0.931	0.860	0.095	0.228	25.22	28.27	0.879	0.883	0.154	0.192
3DGUT	26.31	26.69	0.917	0.817	0.121	0.302	19.00	27.42	0.659	0.866	0.263	0.228
Ours	28.26	28.49	0.922	0.906	0.097	0.190	24.52	28.55	0.874	0.885	0.168	0.220